

# Physlib Monthly Report

1–31 August 2024

**Contributors:** Joseph Tooby-Smith, Pietro Monticone

Generated 14 August 2026 from commit 86b3b8a.

## Abstract

Physlib is a community library of formalised physics for the [Lean 4](#) proof assistant: definitions and results from across physics, stated in a formal language and checked by machine. This report is an automatically generated record of one month’s additions to it.

It covers 1–31 August 2024 on branch `master` of [leanprover-community/physlib](#), between commits `2439ec3` and `86b3b8a`. Over that period 2 contributors landed 132 commits, changing 10,481 lines across 96 files and adding 582 new Lean declarations, each catalogued with its statement in Section 2. The complete diff is available at [GitHub compare](#).

## Contents

|          |                                                                                          |          |
|----------|------------------------------------------------------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                                                                      | <b>2</b> |
| 1.1      | Contributors . . . . .                                                                   | 2        |
| 1.2      | Reviewers . . . . .                                                                      | 2        |
| 1.3      | Modified subfolders . . . . .                                                            | 2        |
| 1.4      | New declarations by kind . . . . .                                                       | 2        |
| <b>2</b> | <b>Details by file</b>                                                                   | <b>3</b> |
| 2.1      | <code>HepLean.SpaceTime.LorentzTensor.Basic</code> . . . . .                             | 3        |
| 2.2      | <code>HepLean.SpaceTime.LorentzTensor.Contraction</code> . . . . .                       | 5        |
| 2.3      | <code>HepLean.SpaceTime.LorentzTensor.EinsteinNotation.Basic</code> . . . . .            | 7        |
| 2.4      | <code>HepLean.SpaceTime.LorentzTensor.EinsteinNotation.IndexNotation</code> . . . . .    | 8        |
| 2.5      | <code>HepLean.SpaceTime.LorentzTensor.IndexNotation.Basic</code> . . . . .               | 10       |
| 2.6      | <code>HepLean.SpaceTime.LorentzTensor.IndexNotation.ColorIndexList.Append</code> .       | 13       |
| 2.7      | <code>HepLean.SpaceTime.LorentzTensor.IndexNotation.ColorIndexList.Basic</code> . .      | 15       |
| 2.8      | <code>HepLean.SpaceTime.LorentzTensor.IndexNotation.ColorIndexList.Contraction</code> 16 |          |
| 2.9      | <code>HepLean.SpaceTime.LorentzTensor.IndexNotation.ColorIndexList.ContrPerm</code> 18   |          |
| 2.10     | <code>HepLean.SpaceTime.LorentzTensor.IndexNotation.IndexList.Basic</code> . . . . .     | 22       |
| 2.11     | <code>HepLean.SpaceTime.LorentzTensor.IndexNotation.IndexList.Color</code> . . . . .     | 26       |
| 2.12     | <code>HepLean.SpaceTime.LorentzTensor.IndexNotation.IndexList.Contraction</code> .       | 32       |
| 2.13     | <code>HepLean.SpaceTime.LorentzTensor.IndexNotation.IndexList.CountId</code> . . .       | 35       |
| 2.14     | <code>HepLean.SpaceTime.LorentzTensor.IndexNotation.IndexList.Duals</code> . . . . .     | 39       |
| 2.15     | <code>HepLean.SpaceTime.LorentzTensor.IndexNotation.IndexList.Equivs</code> . . . . .    | 46       |
| 2.16     | <code>HepLean.SpaceTime.LorentzTensor.IndexNotation.IndexList.Normalize</code> . .       | 49       |
| 2.17     | <code>HepLean.SpaceTime.LorentzTensor.IndexNotation.IndexList.OnlyUniqueDuals</code> 55  |          |
| 2.18     | <code>HepLean.SpaceTime.LorentzTensor.IndexNotation.IndexList.Subperm</code> . . .       | 57       |
| 2.19     | <code>HepLean.SpaceTime.LorentzTensor.IndexNotation.IndexString</code> . . . . .         | 57       |
| 2.20     | <code>HepLean.SpaceTime.LorentzTensor.IndexNotation.TensorIndex</code> . . . . .         | 59       |
| 2.21     | <code>HepLean.SpaceTime.LorentzTensor.Real.Basic</code> . . . . .                        | 66       |

|      |                                                                              |    |
|------|------------------------------------------------------------------------------|----|
| 2.22 | <a href="#">HepLean.SpaceTime.LorentzTensor.Real.IndexNotation</a> . . . . . | 67 |
| 2.23 | <a href="#">HepLean.SpaceTime.LorentzTensor.RisingLowering</a> . . . . .     | 68 |

# 1 Introduction

During Aug 2024, 2 contributors submitted 132 commits that changed 10,481 lines (+8,051 / -2,430) across 96 files spanning 13 top-level areas of the repository. Of those, 21 files were newly added and 2 were removed outright. This report catalogues 582 newly added Lean declarations: 421 lemmas, 119 defs, 35 instances, 4 structures, 1 class, 1 abbrev, and 1 inductive.

See the [full compare diff](#) and the [list of commits](#) on GitHub for the raw source.

## 1.1 Contributors

The following individuals authored commits merged during this reporting period, listed alphabetically.

| Contributor                        | Lines Changed | Commits |
|------------------------------------|---------------|---------|
| <a href="#">Joseph Tooby-Smith</a> | 41,236        | 118     |
| <a href="#">Pietro Monticone</a>   | 335           | 14      |

## 1.2 Reviewers

The following individuals approved pull requests during this reporting period, listed alphabetically.

*No reviewers identified for this month.*

## 1.3 Modified subfolders

| Subfolder                   | Files | New | Deleted | Additions | Deletions |
|-----------------------------|-------|-----|---------|-----------|-----------|
| HepLean/SpaceTime           | 43    | +21 | -2      | +6,527    | -752      |
| HepLean/AnomalyCancellation | 29    | —   | —       | +804      | -953      |
| HepLean/FlavorPhysics       | 6     | —   | —       | +465      | -474      |
| HepLean/Mathematics         | 2     | —   | —       | +46       | -91       |
| docs                        | 2     | —   | —       | +62       | -37       |
| HepLean/FeynmanDiagrams     | 1     | —   | —       | +31       | -32       |
| HepLean/StandardModel       | 4     | —   | —       | +26       | -33       |
| README.md                   | 1     | —   | —       | +27       | -26       |
| lake-manifest.json          | 1     | —   | —       | +24       | -14       |
| scripts                     | 4     | —   | —       | +15       | -13       |
| HepLean.lean                | 1     | —   | —       | +21       | -2        |
| lakefile.toml               | 1     | —   | —       | +2        | -2        |
| lean-toolchain              | 1     | —   | —       | +1        | -1        |

## 1.4 New declarations by kind

| Kind      | Count |
|-----------|-------|
| lemma     | 421   |
| def       | 119   |
| instance  | 35    |
| structure | 4     |
| class     | 1     |

|           |   |
|-----------|---|
| abbrev    | 1 |
| inductive | 1 |

## 2 Details by file

### 2.1 HepLean.SpaceTime.LorentzTensor.Basic

[HepLean/SpaceTime/LorentzTensor/Basic.lean](#) on GitHub.

#### structure TensorColor

[\[source\]](#)

*The index color data associated with a tensor structure. This corresponds to a type with an involution.*

```
structure TensorColor where
  Color : Type
  τ : Color → Color
  τ_involutive : Function.Involutive τ
```

#### def colorRel

[\[source\]](#)

*A relation on colors which is true if the two colors are equal or are duals.*

```
def colorRel (μ ν : C.Color) : Prop
```

#### instance Decidable (colorRel C μ ν)

[\[source\]](#)

```
instance : Decidable (colorRel C μ ν)
```

#### lemma colorRel\_equivalence

[\[source\]](#)

*An equivalence relation on colors which is true if the two colors are equal or are duals.*

```
lemma colorRel_equivalence : Equivalence C.colorRel where
```

#### instance colorSetoid

[\[source\]](#)

*The structure of a setoid on colors, two colors are related if they are equal, or dual.*

```
instance colorSetoid : Setoid C.Color
```

#### def colorQuot

[\[source\]](#)

*A map taking a color to its equivalence class in colorSetoid.*

```
def colorQuot (μ : C.Color) : Quotient C.colorSetoid
```

#### instance Decidable (μ ≈ ν)

[\[source\]](#)

```
instance (μ ν : C.Color) : Decidable (μ ≈ ν)
```

**instance DecidableEq (Quotient  $\mathcal{C}$ .colorSetoid)**

[\[source\]](#)

**instance** : DecidableEq (Quotient  $\mathcal{C}$ .colorSetoid)

**def ColorMap**

[\[source\]](#)

*The types of maps from an  $X$  to  $\mathcal{C}$ .Color.*

**def** ColorMap (X : Type)

**def MapIso**

[\[source\]](#)

*A relation, given an equivalence of types, between ColorMap which is true if related by composition of the equivalence.*

**def** MapIso (e :  $X \approx Y$ ) (cX : ColorMap  $\mathcal{C}$  X) (cY : ColorMap  $\mathcal{C}$  Y) : Prop

**def ColorMap.sum**

[\[source\]](#)

*The sum of two color maps, formed by Sum.elim.*

**def** sum (cX : ColorMap  $\mathcal{C}$  X) (cY : ColorMap  $\mathcal{C}$  Y) : ColorMap  $\mathcal{C}$  (Sum X Y)

**def ColorMap.dual**

[\[source\]](#)

*The dual of a color map, formed by composition with  $\mathcal{C}.\tau$ .*

**def** dual (cX : ColorMap  $\mathcal{C}$  X) : ColorMap  $\mathcal{C}$  X

**lemma symm**

[\[source\]](#)

**lemma** symm (h : cX.MapIso e cY) : cY.MapIso e.symm cX

**lemma symm'**

[\[source\]](#)

**lemma** symm' : cX.MapIso e cY  $\leftrightarrow$  cY.MapIso e.symm cX

**lemma trans**

[\[source\]](#)

**lemma** trans (h : cX.MapIso e cY) (h' : cY.MapIso e' cZ) :  
cX.MapIso (e.trans e') cZ

**lemma MapIso.sum**

[\[source\]](#)

**lemma** sum {eX :  $X \approx X'$ } {eY :  $Y \approx Y'$ } (hX : cX.MapIso eX cX') (hY : cY.MapIso eY cY') :  
(cX.sum cY).MapIso (eX.sumCongr eY) (cX'.sum cY')

**lemma MapIso.dual**

[\[source\]](#)

**lemma** dual {e :  $X \approx Y$ } (h : cX.MapIso e cY) :  
cX.dual.MapIso e cY.dual

**lemma mapIso\_ext**[\[source\]](#)

```
lemma mapIso_ext {c :  $\mathcal{T}$ .ColorMap X} {d :  $\mathcal{T}$ .ColorMap Y} (e e' : X  $\approx$  Y) (h : c.MapIso e d)
  (h' : c.MapIso e' d) (he : e = e') :  $\mathcal{T}$ .mapIso e h =  $\mathcal{T}$ .mapIso e' h'
```

**lemma tensoratorEquiv\_symm\_tprod**[\[source\]](#)

```
@[simp]
lemma tensoratorEquiv_symm_tprod (f :  $\mathcal{T}$ .PureTensor (Sum.elim cX cY)) :
  ( $\mathcal{T}$ .tensoratorEquiv cX cY).symm ((PiTensorProduct.tprod R) f) =
  (PiTensorProduct.tprod R) ( $\mathcal{T}$ .inlPureTensor f)  $\otimes_{\mathcal{T}[R]}$ 
  (PiTensorProduct.tprod R) ( $\mathcal{T}$ .inrPureTensor f)
```

**def isEmptyEquiv**[\[source\]](#)

*The equivalence between  $\mathcal{T}$ .Tensor cX and R when the indexing set X is empty.*

```
def isEmptyEquiv [IsEmpty X] :  $\mathcal{T}$ .Tensor cX  $\approx_l[R]$  R
```

**lemma isEmptyEquiv\_tprod**[\[source\]](#)

```
@[simp]
lemma isEmptyEquiv_tprod [IsEmpty X] (f :  $\mathcal{T}$ .PureTensor cX) :
   $\mathcal{T}$ .isEmptyEquiv (PiTensorProduct.tprod R f) = 1
```

**2.2 HepLean.SpaceTime.LorentzTensor.Contraction**

[HepLean/SpaceTime/LorentzTensor/Contraction.lean](#) on GitHub.

**def ContrAll**[\[source\]](#)

*Given an equivalence e of types the condition that the color map cX is the dual to cY up to this equivalence.*

```
def ContrAll (e : X  $\approx$  Y) (cX : ColorMap  $\mathcal{C}$  X) (cY : ColorMap  $\mathcal{C}$  Y) : Prop
```

**lemma toMapIso**[\[source\]](#)

```
lemma toMapIso (h : cX.ContrAll e cY) : cX.MapIso e cY.dual
```

**lemma symm**[\[source\]](#)

```
lemma symm (h : cX.ContrAll e cY) : cY.ContrAll e.symm cX
```

**lemma trans\_mapIso**[\[source\]](#)

```
lemma trans_mapIso {e : X  $\approx$  Y} {e' : Z  $\approx$  Y}
  (h : cX.ContrAll e cY) (h' : cZ.MapIso e' cY) : cX.ContrAll (e.trans e'.symm) cZ
```

**lemma mapIso\_trans**[\[source\]](#)

```

lemma mapIso_trans {e : X ≈ Y} {e' : Z ≈ X}
  (h : cX.ContrAll e cY) (h' : cZ.MapIso e' cX) : cZ.ContrAll (e'.trans e) cY

```

**def contr**[\[source\]](#)

*Given an equivalence  $(C \otimes C) \otimes P \approx X$  the restriction of a color map  $cX$  on to  $P$ .*

```

def contr (e : (C ⊗ C) ⊗ P ≈ X) (cX : ColorMap C X) : ColorMap C P

```

**def contrLeft**[\[source\]](#)

*Given an equivalence  $(C \otimes C) \otimes P \approx X$  the restriction of a color map  $cX$  on  $X$  to the first  $C$ .*

```

def contrLeft (e : (C ⊗ C) ⊗ P ≈ X) (cX : ColorMap C X) : ColorMap C C

```

**def contrRight**[\[source\]](#)

*Given an equivalence  $(C \otimes C) \otimes P \approx X$  the restriction of a color map  $cX$  on  $X$  to the second  $C$ .*

```

def contrRight (e : (C ⊗ C) ⊗ P ≈ X) (cX : ColorMap C X) : ColorMap C C

```

**def ContrCond**[\[source\]](#)

*Given an equivalence  $(C \otimes C) \otimes P \approx X$  the condition on  $cX$  so that we contract the indices of the  $C$ 's under this equivalence.*

```

def ContrCond (e : (C ⊗ C) ⊗ P ≈ X) (cX : ColorMap C X) : Prop

```

**lemma to\_contrAll**[\[source\]](#)

```

lemma to_contrAll (h : cX.ContrCond e) :
  (cX.contrLeft e).ContrAll (Equiv.refl _) (cX.contrRight e)

```

**lemma pairProd\_tmul\_tprod\_tprod**[\[source\]](#)

```

lemma pairProd_tmul_tprod_tprod (fx : (i : X) → T.ColorModule (cX i))
  (fx2 : (i : X) → T.ColorModule (cX2 i)) :
  T.pairProd (PiTensorProduct.tprod R fx ⊗t[R] PiTensorProduct.tprod R fx2) =
  PiTensorProduct.tprod R (fun x => fx x ⊗t[R] fx2 x)

```

**lemma contrAll'\_tmul\_tprod\_tprod**[\[source\]](#)

```

lemma contrAll'_tmul_tprod_tprod (fx : (i : X) → T.ColorModule (cX i))
  (fy : (i : X) → T.ColorModule (T.τ (cX i))) :
  T.contrAll' (PiTensorProduct.tprod R fx ⊗t[R] PiTensorProduct.tprod R fy) =
  (PiTensorProduct.lift (MultilinearMap.mkPiAlgebra R X R))
  (PiTensorProduct.tprod R (fun x => T.contrDual (cX x) (fx x ⊗t[R] fy x)))

```

**lemma contrAll'\_isEmpty\_tmul**[\[source\]](#)

@[simp]

```
lemma contrAll'_isEmpty_tmul [IsEmpty X] (x :  $\mathcal{T}$ .Tensor cX) (y :  $\mathcal{T}$ .Tensor ( $\mathcal{T}.\tau \circ cX$ )) :
   $\mathcal{T}.$ contrAll' (x  $\otimes_{\mathcal{T}}$  y) =  $\mathcal{T}.$ isEmptyEquiv x *  $\mathcal{T}.$ isEmptyEquiv y
```

**lemma contrAll\_tmul**[\[source\]](#)

```
lemma contrAll_tmul (e : X  $\approx$  Y) (h : cX.ContrAll e cY) (x :  $\mathcal{T}$ .Tensor cX) (y :  $\mathcal{T}$ .Tensor cY) :
   $\mathcal{T}.$ contrAll e h (x  $\otimes_{\mathcal{T}}$  y) =  $\mathcal{T}.$ contrAll' (x  $\otimes_{\mathcal{T}}$  [R] (( $\mathcal{T}.$ mapIso e.symm h.symm.toMapIso) y))
```

**lemma contr\_tprod**[\[source\]](#)

```
lemma contr_tprod (e : (C  $\otimes$  C)  $\otimes$  P  $\approx$  X) (h : cX.ContrCond e) (f : (i : X)  $\rightarrow$   $\mathcal{T}.$ ColorModule (cX i)) :
   $\mathcal{T}.$ contr e h (tprod R f) = ( $\mathcal{T}.$ contrAll (Equiv.refl C) h.to_contrAll
    (tprod R (fun i => f (e (Sum.inl (Sum.inl i)))))  $\otimes_{\mathcal{T}}$  [R]
    tprod R (fun i => f (e (Sum.inl (Sum.inr i))))) •
    tprod R (fun (p : P) => f (e (Sum.inr p)))
```

**lemma contr\_tprod\_isEmpty**[\[source\]](#)

@[simp]

```
lemma contr_tprod_isEmpty [IsEmpty C] (e : (C  $\otimes$  C)  $\otimes$  P  $\approx$  X) (h : cX.ContrCond e)
  (f : (i : X)  $\rightarrow$   $\mathcal{T}.$ ColorModule (cX i)) :
   $\mathcal{T}.$ contr e h (tprod R f) = tprod R (fun (p : P) => f (e (Sum.inr p)))
```

**2.3 HepLean.SpaceTime.LorentzTensor.EinsteinNotation.Basic**

[HepLean/SpaceTime/LorentzTensor/EinsteinNotation/Basic.lean](#) on GitHub.

**def einsteinTensorColor**[\[source\]](#)

*Einstein tensors have only one color, corresponding to a down index. .*

```
def einsteinTensorColor : TensorColor where
```

**instance Fintype einsteinTensorColor.Color**[\[source\]](#)

```
instance : Fintype einsteinTensorColor.Color
```

**instance DecidableEq einsteinTensorColor.Color**[\[source\]](#)

```
instance : DecidableEq einsteinTensorColor.Color
```

**def einsteinTensor**[\[source\]](#)

*The TensorStructure associated with n-dimensional tensors.*

```
noncomputable def einsteinTensor (R : Type) [CommSemiring R] (n :  $\mathbb{N}$ ) : TensorStructure R where
```

**instance OfNat einsteinTensorColor.Color 0**

[\[source\]](#)

**instance** : OfNat einsteinTensorColor.Color 0

**instance OfNat (einsteinTensor R n).Color 0**

[\[source\]](#)

**instance** : OfNat (einsteinTensor R n).Color 0

**lemma ofNat\_inst\_eq**

[\[source\]](#)

@[simp]

**lemma** ofNat\_inst\_eq : @einsteinTensor.instOfNatColorOfNatNat R \_ n =  
einsteinTensor.instOfNatColorEinsteinTensorColorOfNatNat

**def toVec**

[\[source\]](#)

*A vector from an Einstein tensor with one index.*

**def** toVec : (einsteinTensor R n).Tensor ![Unit.unit]  $\approx_l$  [R] Fin n  $\rightarrow$  R

**def toMatrix**

[\[source\]](#)

*A matrix from an Einstein tensor with two indices.*

**def** toMatrix : (einsteinTensor R n).Tensor ![Unit.unit, Unit.unit]  $\approx_l$  [R] Matrix (Fin n) (Fin n)  
R

## 2.4 HepLean.SpaceTime.LorentzTensor.EinsteinNotation.IndexNotation

[HepLean/SpaceTime/LorentzTensor/EinsteinNotation/IndexNotation.lean](#) on GitHub.

**instance IndexNotation einsteinTensorColor.Color**

[\[source\]](#)

**instance** : IndexNotation einsteinTensorColor.Color **where**

**instance IndexNotation (einsteinTensor R n).Color**

[\[source\]](#)

**instance** : IndexNotation (einsteinTensor R n).Color

**instance DecidableEq (einsteinTensor R n).Color**

[\[source\]](#)

**instance** : DecidableEq (einsteinTensor R n).Color

**lemma indexNotation\_eq\_color**

[\[source\]](#)

@[simp]

**lemma** indexNotation\_eq\_color : @einsteinTensor.instIndexNotationColor R \_ n =  
instIndexNotationColorEinsteinTensorColor

**lemma decidableEq\_eq\_color**[\[source\]](#)

```
@[simp]
lemma decidableEq_eq_color : @einsteinTensor.instDecidableEqColor R _ n =
  instDecidableEqColorEinsteinTensorColor
```

**lemma einsteinTensor\_color**[\[source\]](#)

```
@[simp]
lemma einsteinTensor_color : (einsteinTensor R n).Color = einsteinTensorColor.Color
```

**lemma toTensorColor\_eq**[\[source\]](#)

```
@[simp]
lemma toTensorColor_eq : (einsteinTensor R n).toTensorColor = einsteinTensorColor
```

**def fromIndexStringColor**[\[source\]](#)

*The construction of a tensor index from a tensor and a string satisfying conditions which can be automatically checked. This is a modified version of `TensorStructure.TensorIndex.mkDualMap` specific to real Lorentz tensors.*

```
noncomputable def fromIndexStringColor {R : Type} [CommSemiring R]
  {cn : Fin n → einsteinTensorColor.Color}
  (T : (einsteinTensor R m).Tensor cn) (s : String)
  (hs : listCharIsIndexString einsteinTensorColor.Color s.toList = true)
  (hn : n = (toIndexList' s hs).length)
  (hD : (toIndexList' s hs).OnlyUniqueDuals)
  (hC : IndexList.ColorCond.bool (toIndexList' s hs))
  (hd : TensorColor.ColorMap.DualMap.boolFin'
    (toIndexList' s hs).colorMap (cn ∘ Fin.cast hn.symm)) :
  (einsteinTensor R m).TensorIndex
```

**lemma fromIndexStringColor\_indexList**[\[source\]](#)

```
@[simp]
lemma fromIndexStringColor_indexList {R : Type} [CommSemiring R]
  {cn : Fin n → einsteinTensorColor.Color}
  (T : (einsteinTensor R m).Tensor cn) (s : String)
  (hs : listCharIsIndexString einsteinTensorColor.Color s.toList = true)
  (hn : n = (toIndexList' s hs).length)
  (hD : (toIndexList' s hs).OnlyUniqueDuals)
  (hC : IndexList.ColorCond.bool (toIndexList' s hs))
  (hd : TensorColor.ColorMap.DualMap.boolFin'
    (toIndexList' s hs).colorMap (cn ∘ Fin.cast hn.symm)) :
  (fromIndexStringColor T s hs hn hD hC hd).toIndexList = toIndexList' s hs
```

**lemma mem\_fin\_list**[\[source\]](#)

```
lemma mem_fin_list (n : ℕ) (i : Fin n) : i ∈ Fin.list n
```

**instance Decidable (i ∈ Fin.list n)**[\[source\]](#)

```
instance (n : ℕ) (i : Fin n) : Decidable (i ∈ Fin.list n)
```

## 2.5 HepLean.SpaceTime.LorentzTensor.IndexNotation.Basic

[HepLean/SpaceTime/LorentzTensor/IndexNotation/Basic.lean](#) on GitHub.

**class IndexNotation** [\[source\]](#)

*The class defining index notation on a type  $X$ . Normally  $X$  will be taken as the type of colors of a `TensorStructure`.*

```
class IndexNotation (X : Type) where
  charList : Finset Char
  notaEquiv : X ≈ charList
```

**def nota** [\[source\]](#)

*The map taking a color to its notation character.*

```
def nota {X : Type} [IndexNotation X] (x : X) : Char
```

**def isNotationChar** [\[source\]](#)

*A character is a notation character if it is in `charList`.*

```
def isNotationChar (c : Char) : Bool
```

**def isNumericSuperscript** [\[source\]](#)

*A character is a numeric superscript if it is e.g.  $^0$ ,  $^1$ , etc.*

```
def isNumericSuperscript (c : Char) : Bool
```

**def IsIndexId** [\[source\]](#)

*Given a character  $f$  which is a notation character, this is true if  $c$  is a subscript when  $f$  is a subscript or  $c$  is a superscript when  $f$  is a superscript.*

```
def IsIndexId (f : Char) (c : Char) : Bool
```

**def listCharIndexTail** [\[source\]](#)

*The proposition for a list of characters to be the tail of an index e.g. `['1', '2', ...]`*

```
def listCharIndexTail (f : Char) (l : List Char) : Prop
```

**instance Decidable (listCharIndexTail f l)** [\[source\]](#)

```
instance : Decidable (listCharIndexTail f l)
```

**def listCharIndex** [\[source\]](#)

*The proposition for a list of characters to be the characters of an index e.g. `['u', '1', '2', ...]`*

```
def listCharIndex (l : List Char) : Prop
```

**lemma listCharIndex\_iff**

[\[source\]](#)

*An auxillary rewrite lemma to prove that listCharIndex is decidable.*

```
lemma listCharIndex_iff (l : List Char) : listCharIndex X l
↔ (if h : l = [] then True else
  let sfst := l.head h
  if ¬ isNotationChar X sfst then False
  else listCharIndexTail sfst l.tail)
```

**instance Decidable (listCharIndex X l)**

[\[source\]](#)

```
instance : Decidable (listCharIndex X l)
```

**def Index**

[\[source\]](#)

*An index for  $X$  is an pair of an element of  $X$  (the color of the index) and a natural number (the id of the index).*

```
def Index : Type
```

**instance DecidableEq (Index X)**

[\[source\]](#)

```
instance : DecidableEq (Index X)
```

**def Index.toColor**

[\[source\]](#)

*The color associated to an index.*

```
def toColor (I : Index X) : X
```

**def Index.id**

[\[source\]](#)

*The natural number representating the id of an index.*

```
def id (I : Index X) : ℕ
```

**lemma eq\_iff\_color\_eq\_and\_id\_eq**

[\[source\]](#)

```
lemma eq_iff_color_eq_and_id_eq (I J : Index X) : I = J ↔ I.toColor = J.toColor ∧ I.id = J.id
```

**def IndexRep**

[\[source\]](#)

*An index rep is a non-empty string satisfying the condition listCharIndex, e.g.  $u_{12}$  or  $u_{43}$  etc.*

```
def IndexRep : Type
```

**instance DecidableEq (IndexRep X)**

[\[source\]](#)

**instance** : DecidableEq (IndexRep X)

**def ofCharList**

[\[source\]](#)

*Creates an index from a non-empty list of characters satisfying listCharIndex.*

**def** ofCharList (l : List Char) (h : listCharIndex X l ∧ l ≠ []) : IndexRep X

**instance ToString (IndexRep X)**

[\[source\]](#)

**instance** : ToString (IndexRep X)

**def head**

[\[source\]](#)

*Gets the first character in an index e.g. <sup>u</sup> as an element of charList X.*

**def** head (s : IndexRep X) : charList X

**def IndexRep.toColor**

[\[source\]](#)

*The color associated to an index.*

**def** toColor (s : IndexRep X) : X

**def charToNat**

[\[source\]](#)

*A map from super and subscript numerical characters to the natural numbers, returning 0 on all other characters.*

**def** charToNat (c : Char) : Nat

**def tail**

[\[source\]](#)

*The numerical characters associated with an index.*

**def** tail (s : IndexRep X) : List Char

**def tailNat**

[\[source\]](#)

*The natural numbers associated with an index.*

**def** tailNat (s : IndexRep X) : List ℕ

**def IndexRep.id**

[\[source\]](#)

*The id of an index, as a natural number.*

**def** id (s : IndexRep X) : ℕ

**def toIndex**[\[source\]](#)

*The index associated with a IndexRep.*

```
def toIndex (s : IndexRep X) : Index X
```

**2.6 HepLean.SpaceTime.LorentzTensor.IndexNotation.ColorIndexList.Append**

[HepLean/SpaceTime/LorentzTensor/IndexNotation/ColorIndexList/Append.lean](#) on GitHub.

**def AppendCond**[\[source\]](#)

*The condition on the ColorIndexLists l and l2 so that on appending they form a ColorIndexList.*

*Note: AppendCond does not form an equivalence relation as it is not reflexive or transitive.*

```
def AppendCond : Prop
```

**def append**[\[source\]](#)

*Given two ColorIndexLists satisfying AppendCond. The corresponding combined ColorIndexList.*

```
def append (h : AppendCond l l2) : ColorIndexList C where
```

**lemma append\_toIndexList**[\[source\]](#)

```
@[simp]
lemma append_toIndexList (h : AppendCond l l2) :
  (l ++[h] l2).toIndexList = l.toIndexList ++ l2.toIndexList
```

**lemma symm**[\[source\]](#)

```
@[symm]
lemma symm (h : AppendCond l l2) : AppendCond l2 l
```

**lemma inr**[\[source\]](#)

```
lemma inr (h : AppendCond l l2) (h' : AppendCond (l ++[h] l2) l3) :
  AppendCond l2 l3
```

**lemma assoc**[\[source\]](#)

```
lemma assoc (h : AppendCond l l2) (h' : AppendCond (l ++[h] l2) l3) :
  AppendCond l (l2 ++[h.inr h'] l3)
```

**lemma swap**[\[source\]](#)

```
lemma swap (h : AppendCond l l2) (h' : AppendCond (l ++[h] l2) l3) :
  AppendCond (l2 ++[h.symm] l) l3
```

**lemma contr\_left**[\[source\]](#)

*If AppendCond l l2 then AppendCond l.contr l2. Note that the inverse is generally not true.*

```
lemma contr_left (h : AppendCond l l2) : AppendCond l.contr l2
```

**lemma contr\_right**[\[source\]](#)

```
lemma contr_right (h : AppendCond l l2) : AppendCond l l2.contr
```

**lemma contr**[\[source\]](#)

```
lemma contr (h : AppendCond l l2) : AppendCond l.contr l2.contr
```

**def bool**[\[source\]](#)

*A boolean which is true for two index lists l and l2 if on appending they can form a ColorIndexList.*

```
def bool (l l2 : IndexList C.Color) : Bool
```

**lemma bool\_iff**[\[source\]](#)

```
lemma bool_iff (l l2 : IndexList C.Color) :  
  bool l l2 ↔ (l ++ l2).withUniqueDual = (l ++ l2).withDual ∧ ColorCond.bool (l ++ l2)
```

**lemma iff\_bool**[\[source\]](#)

```
lemma iff_bool (l l2 : ColorIndexList C) : AppendCond l l2 ↔ bool l.toIndexList l2.toIndexList
```

**lemma countId\_contr\_fst\_eq\_zero\_mem\_snd**[\[source\]](#)

```
lemma countId_contr_fst_eq_zero_mem_snd (h : AppendCond l l2) {I : Index C.Color}  
  (hI : I ∈ l2.val) : l.contr.countId I = 0 ↔ l.countId I = 0
```

**lemma countId\_contr\_snd\_eq\_zero\_mem\_fst**[\[source\]](#)

```
lemma countId_contr_snd_eq_zero_mem_fst (h : AppendCond l l2) {I : Index C.Color}  
  (hI : I ∈ l.val) : l2.contr.countId I = 0 ↔ l2.countId I = 0
```

**lemma append\_contr\_left**[\[source\]](#)

```
lemma append_contr_left (h : AppendCond l l2) :  
  (l.contr ++ [h.contr_left] l2).contr = (l ++ [h] l2).contr
```

**lemma append\_contr\_right**[\[source\]](#)

```
lemma append_contr_right (h : AppendCond l l2) :  
  (l ++ [h.contr_right] l2.contr).contr = (l ++ [h] l2).contr
```

**lemma append\_contr**[\[source\]](#)

```
lemma append_contr (h : AppendCond l l2) :
  (l.contr ++[h.contr] l2.contr).contr = (l ++[h] l2).contr
```

**2.7 HepLean.SpaceTime.LorentzTensor.IndexNotation.ColorIndexList.Basic**

[HepLean/SpaceTime/LorentzTensor/IndexNotation/ColorIndexList/Basic.lean](#) on GitHub.

**structure ColorIndexList**[\[source\]](#)

*A list of indices with the additional constraint that if a index has a dual, that dual is unique, and the dual of an index has dual color to that index.*

*This is the permissible type of indices which can be used for a tensor. For example, the index list [' $u_1$ ', ' $_{u1}$ '] can be extended to a ColorIndexList but the index list [' $u_1$ ', ' $_{u1}$ ', ' $_{u1}$ '] cannot.*

```
structure ColorIndexList (C : TensorColor) [IndexNotation C.Color] extends IndexList C.Color
  where
    unique_duals : toIndexList.OnlyUniqueDuals
    dual_color : IndexList.ColorCond toIndexList
```

**instance Coe (ColorIndexList C) (IndexList C.Color)**[\[source\]](#)

```
instance : Coe (ColorIndexList C) (IndexList C.Color)
```

**def colorMap'**[\[source\]](#)

*The colorMap of a ColorIndexList as a C.ColorMap. This is to be compared with colorMap which is a map  $\text{Fin } l.\text{length} \rightarrow C.\text{Color}$ .*

```
def colorMap' : C.ColorMap (Fin l.length)
```

**lemma ext**[\[source\]](#)

```
@[ext]
lemma ext {l l' : ColorIndexList C} (h : l.val = l'.val) : l = l'
```

**lemma ext'**[\[source\]](#)

```
lemma ext' {l l' : ColorIndexList C} (h : l.toIndexList = l'.toIndexList) : l = l'
```

**lemma orderEmbOffFin\_univ**[\[source\]](#)

```
lemma orderEmbOffFin_univ (n m : ℕ) (h : n = m) :
  Finset.orderEmbOffFin (Finset.univ : Finset (Fin n)) (by simp [h]: Finset.univ.card = m) =
  (Fin.castOrderIso h.symm).toOrderEmbedding
```

**def empty**[\[source\]](#)

*The ColorIndexList whose underlying list of indices is empty.*

```
def empty : ColorIndexList C where
```

```
lemma countId_le_two
```

[\[source\]](#)

```
lemma countId_le_two (l : ColorIndexList C) (I : Index C.Color) : l.countId I ≤ 2
```

## 2.8 HepLean.SpaceTime.LorentzTensor.IndexNotation.ColorIndexList.Contraction

[HepLean/SpaceTime/LorentzTensor/IndexNotation/ColorIndexList/Contraction.lean](#) on GitHub.

```
def contr
```

[\[source\]](#)

*The contraction of a ColorIndexList, l. That is, the ColorIndexList obtained by taking only those indices in l which do not have a dual. This can be thought of as contracting all of those indices with a dual.*

```
def contr : ColorIndexList C where
```

```
lemma contr_contr
```

[\[source\]](#)

```
@[simp]
```

```
lemma contr_contr : l.contr.contr = l.contr
```

```
lemma contr_contr_idMap
```

[\[source\]](#)

```
@[simp]
```

```
lemma contr_contr_idMap (i : Fin l.contr.contr.length) :  
  l.contr.contr.idMap i = l.contr.idMap (Fin.cast (by simp) i)
```

```
lemma contr_contr_colorMap
```

[\[source\]](#)

```
@[simp]
```

```
lemma contr_contr_colorMap (i : Fin l.contr.contr.length) :  
  l.contr.contr.colorMap i = l.contr.colorMap (Fin.cast (by simp) i)
```

```
lemma contr_of_withDual_empty
```

[\[source\]](#)

```
@[simp]
```

```
lemma contr_of_withDual_empty (h : l.withDual = ∅) :  
  l.contr = l
```

```
lemma contr_getDual?_eq_none
```

[\[source\]](#)

```
@[simp]
```

```
lemma contr_getDual?_eq_none (i : Fin l.contr.length) :  
  l.contr.getDual? i = none
```

```
lemma contr_areDualInSelf
```

[\[source\]](#)

```
@[simp]
lemma contr_areDualInSelf (i j : Fin l.contr.length) :
  l.contr.AreDualInSelf i j ↔ False
```

**lemma contr\_countId\_eq\_zero\_of\_countId\_zero**

[\[source\]](#)

```
lemma contr_countId_eq_zero_of_countId_zero (I : Index C.Color)
  (h : l.countId I = 0) : l.contr.countId I = 0
```

**lemma contr\_countId\_eq\_filter**

[\[source\]](#)

```
lemma contr_countId_eq_filter (I : Index C.Color) :
  l.contr.countId I =
    (l.val.filter (fun J => I.id = J.id)).countP
    (fun i => l.val.countP (fun j => i.id = j.id) = 1)
```

**lemma countId\_conr\_le\_one**

[\[source\]](#)

```
lemma countId_conr_le_one (I : Index C.Color) :
  l.contr.countId I ≤ 1
```

**lemma countId\_conr\_eq\_zero\_iff**

[\[source\]](#)

```
lemma countId_conr_eq_zero_iff (I : Index C.Color) :
  l.contr.countId I = 0 ↔ l.countId I = 0 ∨ l.countId I = 2
```

**def contrEquiv**

[\[source\]](#)

*An equivalence splitting the indices of  $l$  into a sum type of those indices and their duals (with choice determined by the ordering on  $\text{Fin}$ ), and those indices without duals.*

*This equivalence is used to contract the indices of tensors.*

```
def contrEquiv : (l.withUniqueDualLT ⊗ l.withUniqueDualLT) ⊗ Fin l.contr.length ≈ Fin l.length
```

**lemma contrEquiv\_inl\_inl\_isSome**

[\[source\]](#)

```
lemma contrEquiv_inl_inl_isSome (i : l.withUniqueDualLT) :
  (l.getDual? (l.contrEquiv (Sum.inl (Sum.inl i)))) isSome
```

**lemma contrEquiv\_inl\_inr\_eq**

[\[source\]](#)

```
@[simp]
lemma contrEquiv_inl_inr_eq (i : l.withUniqueDualLT) :
  (l.contrEquiv (Sum.inl (Sum.inr i))) =
    (l.getDual? i.1).get (l.contrEquiv_inl_inl_isSome i)
```

**lemma contrEquiv\_inl\_inl\_eq**

[\[source\]](#)

```
@[simp]
lemma contrEquiv_inl_inl_eq (i : l.withUniqueDualLT) :
  (l.contrEquiv (Sum.inl (Sum.inl i))) = i
```

### lemma contrEquiv\_colorMapIso

[\[source\]](#)

```
lemma contrEquiv_colorMapIso :
  ColorMap.MapIso (Equiv.refl (Fin l.contr.length))
  (ColorMap.contr l.contrEquiv l.colorMap) l.contr.colorMap
```

### lemma contrEquiv\_contrCond

[\[source\]](#)

```
lemma contrEquiv_contrCond : ColorMap.ContrCond l.contrEquiv l.colorMap
```

### lemma contrEquiv\_on\_withDual\_empty

[\[source\]](#)

```
@[simp]
lemma contrEquiv_on_withDual_empty (i : Fin l.contr.length) (h : l.withDual = ∅) :
  l.contrEquiv (Sum.inr i) = Fin.cast (by simp [h]) i
```

## 2.9 HepLean.SpaceTime.LorentzTensor.IndexNotation.ColorIndexList.ContrPerm

[HepLean/SpaceTime/LorentzTensor/IndexNotation/ColorIndexList/ContrPerm.lean](#) on GitHub.

### def ContrPerm

[\[source\]](#)

*Two ColorIndexLists are said to be related by contracted permutations, ContrPerm, if and only if:*

*1) Their contractions are the same length. 2) Every index in the contracted list of one has a unique dual in the contracted list of the other and that dual has a the same color.*

```
def ContrPerm : Prop
```

### lemma getDualInOtherEquiv\_eq

[\[source\]](#)

```
lemma getDualInOtherEquiv_eq (h : l.ContrPerm l2) (i : Fin l.contr.length) :
  l2.contr.val.get (l.contr.getDualInOtherEquiv l2.contr (i, by
    rw [h.2.1]
    exact Finset.mem_univ i)).1 = l.contr.val.get i
```

### lemma mem\_snd\_of\_mem\_snd

[\[source\]](#)

```
lemma mem_snd_of_mem_snd (h : l.ContrPerm l2) {I : Index C.Color} (hI : I ∈ l.contr.val) :
  I ∈ l2.contr.val
```

### lemma countSelf\_eq\_one\_snd\_of\_countSelf\_eq\_one\_fst

[\[source\]](#)

```
lemma countSelf_eq_one_snd_of_countSelf_eq_one_fst (h : l.ContrPerm l2) {I : Index C.Color}
  (h1 : l.contr.countSelf I = 1) : l2.contr.countSelf I = 1
```

**lemma getDualInOtherEquiv\_eq\_of\_countSelf**[\[source\]](#)

```

lemma getDualInOtherEquiv_eq_of_countSelf
  (hn : IndexList.Subperm l.contr l2.contr.toIndexList) (i : Fin l.contr.length)
  (h1 : l2.contr.countId (l.contr.val.get i) = 1)
  (h2 : l2.contr.countSelf (l.contr.val.get i) = 1) :
  l2.contr.val.get (l.contr.getDualInOtherEquiv l2.contr (i, by
    rw [hn]
    exact Finset.mem_univ i))).1 = l.contr.val.get i

```

**lemma colorMap\_eq\_of\_countSelf**[\[source\]](#)

```

lemma colorMap_eq_of_countSelf (hn : IndexList.Subperm l.contr l2.contr.toIndexList)
  (h2 : ∀ i, l.contr.countSelf (l.contr.val.get i) = 1
  → l2.contr.countSelf (l.contr.val.get i) = 1) :
  l2.contr.colorMap' ◦ Subtype.val ◦ (l.contr.getDualInOtherEquiv l2.contr)
  = l.contr.colorMap' ◦ Subtype.val

```

**lemma iff\_count\_fin**[\[source\]](#)

```

lemma iff_count_fin : l.ContrPerm l2 ↔
  l.contr.length = l2.contr.length ∧ IndexList.Subperm l.contr l2.contr.toIndexList ∧
  ∀ i, l.contr.countSelf (l.contr.val.get i) = 1 →
  l2.contr.countSelf (l.contr.val.get i) = 1

```

**lemma iff\_count'**[\[source\]](#)

```

lemma iff_count' : l.ContrPerm l2 ↔
  l.contr.length = l2.contr.length ∧ IndexList.Subperm l.contr l2.contr.toIndexList ∧
  ∀ I, l.contr.countSelf I = 1 → l2.contr.countSelf I = 1

```

**lemma iff\_count**[\[source\]](#)

```

lemma iff_count : l.ContrPerm l2 ↔ l.contr.length = l2.contr.length ∧
  ∀ I, l.contr.countSelf I = 1 → l2.contr.countSelf I = 1

```

**lemma symm**[\[source\]](#)

*The relation ContrPerm is symmetric.*

```

@[symm]
lemma symm (h : ContrPerm l l') : ContrPerm l' l

```

**lemma iff\_countSelf**[\[source\]](#)

```

lemma iff_countSelf : l.ContrPerm l2 ↔ ∀ I, l.contr.countSelf I = l2.contr.countSelf I

```

**lemma contr\_perm**[\[source\]](#)

```

lemma contr_perm (h : l.ContrPerm l2) : l.contr.val.Perm l2.contr.val

```

**lemma refl**[\[source\]](#)

*The relation ContrPerm is reflexive.*

```
@[simp]
lemma refl (l : ColorIndexList C) : ContrPerm l l
```

**lemma trans**[\[source\]](#)

*The relation ContrPerm is transitive.*

```
@[trans]
lemma trans (h1 : ContrPerm l l2) (h2 : ContrPerm l2 l3) : ContrPerm l l3
```

**lemma equivalence**[\[source\]](#)

*ContrPerm forms an equivalence relation.*

```
lemma equivalence : Equivalence (@ContrPerm C _ _) where
```

**lemma symm\_trans**[\[source\]](#)

```
lemma symm_trans (h1 : ContrPerm l l2) (h2 : ContrPerm l2 l3) :
  (h1.trans h2).symm = h2.symm.trans h1.symm
```

**lemma contr\_self**[\[source\]](#)

```
@[simp]
lemma contr_self : ContrPerm l l.contr
```

**lemma self\_contr**[\[source\]](#)

```
@[simp]
lemma self_contr : ContrPerm l.contr l
```

**lemma length\_of\_no\_contr**[\[source\]](#)

```
lemma length_of_no_contr (h : l.ContrPerm l') (h1 : l.withDual = ∅) (h2 : l'.withDual = ∅) :
  l.length = l'.length
```

**lemma mem\_withUniqueDualInOther\_of\_no\_contr**[\[source\]](#)

```
lemma mem_withUniqueDualInOther_of_no_contr (h : l.ContrPerm l') (h1 : l.withDual = ∅)
  (h2 : l'.withDual = ∅) : ∀ (x : Fin l.length), x ∈ l.withUniqueDualInOther l'.toIndexList
```

**def contrPermEquiv**[\[source\]](#)

*Given two ColorIndexList related by contracted permutations, the equivalence between the indices of the corresponding contracted index lists. This equivalence is the permutation between the contracted indices.*

```
def contrPermEquiv {l l' : ColorIndexList C} (h : ContrPerm l l') :
  Fin l.contr.length ≈ Fin l'.contr.length
```

**lemma contrPermEquiv\_colorMap\_iso**[\[source\]](#)

```
lemma contrPermEquiv_colorMap_iso {l l' : ColorIndexList C} (h : ContrPerm l l') :
  ColorMap.MapIso (contrPermEquiv h).symm l'.contr.colorMap' l.contr.colorMap'
```

**lemma contrPermEquiv\_colorMap\_iso'**[\[source\]](#)

```
lemma contrPermEquiv_colorMap_iso' {l l' : ColorIndexList C} (h : ContrPerm l l') :
  ColorMap.MapIso (contrPermEquiv h) l.contr.colorMap' l'.contr.colorMap'
```

**lemma contrPermEquiv\_refl**[\[source\]](#)

```
@[simp]
```

```
lemma contrPermEquiv_refl : contrPermEquiv (ContrPerm.refl l) = Equiv.refl _
```

**lemma contrPermEquiv\_symm**[\[source\]](#)

```
@[simp]
```

```
lemma contrPermEquiv_symm {l l' : ColorIndexList C} (h : ContrPerm l l') :
  (contrPermEquiv h).symm = contrPermEquiv h.symm
```

**lemma contrPermEquiv\_trans**[\[source\]](#)

```
@[simp]
```

```
lemma contrPermEquiv_trans {l l2 l3 : ColorIndexList C}
  (h1 : ContrPerm l l2) (h2 : ContrPerm l2 l3) :
  (contrPermEquiv h1).trans (contrPermEquiv h2) = contrPermEquiv (h1.trans h2)
```

**lemma contrPermEquiv\_self\_contr**[\[source\]](#)

```
@[simp]
```

```
lemma contrPermEquiv_self_contr {l : ColorIndexList C} :
  contrPermEquiv (contr_self : ContrPerm l l.contr) =
  (Fin.castOrderIso (by simp)).toEquiv
```

**lemma contrPermEquiv\_contr\_self**[\[source\]](#)

```
@[simp]
```

```
lemma contrPermEquiv_contr_self {l : ColorIndexList C} :
  contrPermEquiv (self_contr : ContrPerm l.contr l) =
  (Fin.castOrderIso (by simp)).toEquiv
```

**def permEquiv**[\[source\]](#)

*Given two ColorIndexList related by permutations and without duals, the equivalence between the indices of the corresponding index lists. This equivalence is the permutation between the indices.*

```
def permEquiv {l l' : ColorIndexList C} (h : ContrPerm l l')
  (h1 : l.withDual = ∅) (h2 : l'.withDual = ∅) : Fin l.length ≈ Fin l'.length
```

**lemma permEquiv\_colorMap\_iso**[\[source\]](#)

```
lemma permEquiv_colorMap_iso {l l' : ColorIndexList C} (h : ContrPerm l l')
  (h1 : l.withDual = ∅) (h2 : l'.withDual = ∅) :
  ColorMap.MapIso (permEquiv h h1 h2).symm l'.colorMap' l.colorMap'
```

**2.10 HepLean.SpaceTime.LorentzTensor.IndexNotation.IndexList.Basic**

[HepLean/SpaceTime/LorentzTensor/IndexNotation/IndexList/Basic.lean](#) on GitHub.

**structure IndexList**[\[source\]](#)

*The type of lists of indices.*

```
structure IndexList where
  val : List (Index X)
```

**def length**[\[source\]](#)

*The number of indices in an index list.*

```
def length : ℕ
```

**lemma ext**[\[source\]](#)

```
lemma ext (h : l.val = l2.val) : l = l2
```

**def cons**[\[source\]](#)

*The index list constructed by prepending an index to the list.*

```
def cons (i : Index X) : IndexList X
```

**lemma cons\_val**[\[source\]](#)

```
@[simp]
lemma cons_val (i : Index X) : (l.cons i).val = i :: l.val
```

**lemma cons\_length**[\[source\]](#)

```
@[simp]
lemma cons_length (i : Index X) : (l.cons i).length = l.length + 1
```

**def tail**[\[source\]](#)

*The tail of an index list. That is, the index list with the first index dropped.*

```
def tail : IndexList X
```

**lemma tail\_val**[\[source\]](#)

```
@[simp]
lemma tail_val : l.tail.val = l.val.tail
```

**def head**[\[source\]](#)

*The first index in a non-empty index list.*

```
def head (h : l ≠ {val := ∅}) : Index X
```

**lemma head\_cons\_tail**[\[source\]](#)

```
lemma head_cons_tail (h : l ≠ {val := ∅}) : l = (l.tail.cons (l.head h))
```

**lemma induction**[\[source\]](#)

```
lemma induction {P : IndexList X → Prop} (h_nil : P {val := ∅})
  (h_cons : ∀ (x : Index X) (xs : IndexList X), P xs → P (xs.cons x)) (l : IndexList X) : P l
```

**def colorMap**[\[source\]](#)

*The map of from  $\text{Fin } s.\text{numIndices}$  into colors associated to an index list.*

```
def colorMap : Fin l.length → X
```

**lemma colorMap\_cast**[\[source\]](#)

```
lemma colorMap_cast {l1 l2 : IndexList X} (h : l1 = l2) :
  l1.colorMap = l2.colorMap ∘ Fin.cast (congrArg length h)
```

**def idMap**[\[source\]](#)

*The map of from  $\text{Fin } s.\text{numIndices}$  into the natural numbers associated to an index list.*

```
def idMap : Fin l.length → Nat
```

**lemma idMap\_cast**[\[source\]](#)

```
lemma idMap_cast {l1 l2 : IndexList X} (h : l1 = l2) (i : Fin l1.length) :
  l1.idMap i = l2.idMap (Fin.cast (by rw [h]) i)
```

**lemma ext\_colorMap\_idMap**[\[source\]](#)

```
lemma ext_colorMap_idMap {l l2 : IndexList X} (hl : l.length = l2.length)
  (hi : l.idMap = l2.idMap ∘ Fin.cast hl) (hc : l.colorMap = l2.colorMap ∘ Fin.cast hl) :
  l = l2
```

**def toPosSet**[\[source\]](#)

*Given a list of indices a subset of  $\text{Fin } l.\text{numIndices} \times \text{Index } X$  of pairs of positions in  $l$  and the corresponding item in  $l$ .*

```
def toPosSet (l : IndexList X) : Set (Fin l.length × Index X)
```

**def toPosSetEquiv**[\[source\]](#)*Equivalence between toPosSet and Fin l.numIndices.*

```
def toPosSetEquiv (l : IndexList X) : l.toPosSet ≈ Fin l.length where
```

**lemma toPosSet\_is\_finite**[\[source\]](#)

```
lemma toPosSet_is_finite (l : IndexList X) : l.toPosSet.Finite
```

**instance Fintype l.toPosSet**[\[source\]](#)

```
instance : Fintype l.toPosSet where
```

**def toPosFinset**[\[source\]](#)*Given a list of indices a finite set of Fin l.length × Index X of pairs of positions in l and the corresponding item in l.*

```
def toPosFinset (l : IndexList X) : Finset (Fin l.length × Index X)
```

**def fromFinMap**[\[source\]](#)*The construction of a list of indices from a map from Fin n to Index X.*

```
def fromFinMap {n : ℕ} (f : Fin n → Index X) : IndexList X where
```

**lemma fromFinMap\_numIndices**[\[source\]](#)

```
@[simp]
lemma fromFinMap_numIndices {n : ℕ} (f : Fin n → Index X) :
  (fromFinMap f).length = n
```

**instance HAppend (IndexList X) (IndexList X) (IndexList X)**[\[source\]](#)

```
instance : HAppend (IndexList X) (IndexList X) (IndexList X) where
```

**lemma cons\_append**[\[source\]](#)

```
@[simp]
lemma cons_append (i : Index X) : (l.cons i) ++ l2 = (l ++ l2).cons i
```

**lemma append\_length**[\[source\]](#)

```
@[simp]
lemma append_length : (l ++ l2).length = l.length + l2.length
```

**lemma append\_assoc**[\[source\]](#)

```
lemma append_assoc : l ++ l2 ++ l3 = l ++ (l2 ++ l3)
```

**def appendEquiv**[\[source\]](#)

*An equivalence between the sum of the types of indices of  $l$  and  $l2$  and the type of indices of the joined index list  $l ++ l2$ .*

```
def appendEquiv {l l2 : IndexList X} : Fin l.length ⊕ Fin l2.length ≈ Fin (l ++ l2).length
```

**def appendInl**[\[source\]](#)

*The inclusion of the indices of  $l$  into the indices of  $l ++ l2$ .*

```
def appendInl : Fin l.length ↪ Fin (l ++ l2).length where
```

**def appendInr**[\[source\]](#)

*The inclusion of the indices of  $l2$  into the indices of  $l ++ l2$ .*

```
def appendInr : Fin l2.length ↪ Fin (l ++ l2).length where
```

**lemma appendInl\_appendEquiv**[\[source\]](#)

```
@[simp]
lemma appendInl_appendEquiv :
  (l.appendInl l2).trans appendEquiv.symm.toEmbedding =
  {toFun := Sum.inl, inj' := Sum.inl_injective}
```

**lemma appendInr\_appendEquiv**[\[source\]](#)

```
@[simp]
lemma appendInr_appendEquiv :
  (l.appendInr l2).trans appendEquiv.symm.toEmbedding =
  {toFun := Sum.inr, inj' := Sum.inr_injective}
```

**lemma append\_val**[\[source\]](#)

```
@[simp]
lemma append_val {l l2 : IndexList X} : (l ++ l2).val = l.val ++ l2.val
```

**lemma idMap\_append\_inl**[\[source\]](#)

```
@[simp]
lemma idMap_append_inl {l l2 : IndexList X} (i : Fin l.length) :
  (l ++ l2).idMap (appendEquiv (Sum.inl i)) = l.idMap i
```

**lemma idMap\_append\_inr**[\[source\]](#)

```
@[simp]
lemma idMap_append_inr {l l2 : IndexList X} (i : Fin l2.length) :
  (l ++ l2).idMap (appendEquiv (Sum.inr i)) = l2.idMap i
```

**lemma colorMap\_append\_inl**[\[source\]](#)

```
@[simp]
lemma colorMap_append_inl {l l2 : IndexList X} (i : Fin l.length) :
  (l ++ l2).colorMap (appendEquiv (Sum.inl i)) = l.colorMap i
```

**lemma colorMap\_append\_inl'**[\[source\]](#)

```
@[simp]
lemma colorMap_append_inl' :
  (l ++ l2).colorMap ∘ appendEquiv ∘ Sum.inl = l.colorMap
```

**lemma colorMap\_append\_inr**[\[source\]](#)

```
@[simp]
lemma colorMap_append_inr {l l2 : IndexList X} (i : Fin l2.length) :
  (l ++ l2).colorMap (appendEquiv (Sum.inr i)) = l2.colorMap i
```

**lemma colorMap\_append\_inr'**[\[source\]](#)

```
@[simp]
lemma colorMap_append_inr' :
  (l ++ l2).colorMap ∘ appendEquiv ∘ Sum.inr = l2.colorMap
```

**lemma colorMap\_sumElim**[\[source\]](#)

```
lemma colorMap_sumElim (l1 l2 : IndexList X) :
  Sum.elim l1.colorMap l2.colorMap =
  (l1 ++ l2).colorMap ∘ appendEquiv
```

**lemma filter\_sort\_comm**[\[source\]](#)

```
lemma filter_sort_comm {n : ℕ} (s : Finset (Fin n)) (p : Fin n → Prop) [DecidablePred p] :
  List.filter p (Finset.sort (fun i j => i ≤ j) s) =
  Finset.sort (fun i j => i ≤ j) (Finset.filter p s)
```

**lemma filter\_id\_eq\_sort**[\[source\]](#)

```
lemma filter_id_eq_sort (i : Fin l.length) : l.val.filter (fun J => (l.val.get i).id = J.id) =
  List.map l.val.get (Finset.sort (fun i j => i ≤ j)
    (Finset.filter (fun j => l.idMap i = l.idMap j) Finset.univ))
```

**2.11 HepLean.SpaceTime.LorentzTensor.IndexNotation.IndexList.Color**

[HepLean/SpaceTime/LorentzTensor/IndexNotation/IndexList/Color.lean](#) on GitHub.

**def dual**[\[source\]](#)

*The dual of an index is the index with the same id, but opposite color.*

```
def dual : Index C.Color
```

**lemma dual\_dual**[\[source\]](#)

```
@[simp]
lemma dual_dual : I.dual.dual = I
```

**lemma dual\_id**[\[source\]](#)

```
@[simp]
lemma dual_id : I.dual.id = I.id
```

**lemma dual\_color**[\[source\]](#)

```
@[simp]
lemma dual_color : I.dual.toColor = C.τ I.toColor
```

**def countColorQuot**[\[source\]](#)

*The number of times I or its dual appears in an IndexList.*

```
def countColorQuot (I : Index C.Color) : ℕ
```

**lemma countColorQuot\_eq\_filter\_id\_countP**[\[source\]](#)

```
lemma countColorQuot_eq_filter_id_countP (I : Index C.Color) :
  l.countColorQuot I =
    (l.val.filter (fun J => I.id = J.id)).countP
    (fun J => I.toColor = J.toColor v I.toColor = C.τ (J.toColor))
```

**lemma countColorQuot\_eq\_filter\_color\_countP**[\[source\]](#)

```
lemma countColorQuot_eq_filter_color_countP (I : Index C.Color) :
  l.countColorQuot I =
    (l.val.filter (fun J => I.toColor = J.toColor v I.toColor = C.τ (J.toColor))).countP
    (fun J => I.id = J.id)
```

**lemma countColorQuot\_append**[\[source\]](#)

```
@[simp]
lemma countColorQuot_append (I : Index C.Color) :
  (l ++ l2).countColorQuot I = countColorQuot l I + countColorQuot l2 I
```

**lemma countColorQuot\_eq\_countId\_iff\_of\_isSome**[\[source\]](#)

```
lemma countColorQuot_eq_countId_iff_of_isSome (hl : l.OnlyUniqueDuals) (i : Fin l.length)
  (hi : (l.getDual? i).isSome) :
  l.countColorQuot (l.val.get i) = l.countId (l.val.get i) ↔
  (l.colorMap i = l.colorMap ((l.getDual? i).get hi) v
   l.colorMap i = C.τ (l.colorMap ((l.getDual? i).get hi)))
```

**lemma countColorQuot\_of\_countId\_zero**[\[source\]](#)

```
lemma countColorQuot_of_countId_zero {I : Index C.Color} (h : l.countId I = 0) :
  l.countColorQuot I = 0
```

**lemma countColorQuot\_le\_countId**[\[source\]](#)

```
lemma countColorQuot_le_countId (I : Index C.Color) :
  l.countColorQuot I ≤ l.countId I
```

**lemma countColorQuot\_contrIndexList\_le\_one**[\[source\]](#)

```
lemma countColorQuot_contrIndexList_le_one (I : Index C.Color) :
  l.contrIndexList.countColorQuot I ≤ 1
```

**lemma countColorQuot\_contrIndexList\_eq\_zero\_or\_one**[\[source\]](#)

```
lemma countColorQuot_contrIndexList_eq_zero_or_one (I : Index C.Color) :
  l.contrIndexList.countColorQuot I = 0 ∨ l.contrIndexList.countColorQuot I = 1
```

**lemma countColorQuot\_contrIndexList\_le\_countColorQuot**[\[source\]](#)

```
lemma countColorQuot_contrIndexList_le_countColorQuot (I : Index C.Color) :
  l.contrIndexList.countColorQuot I ≤ l.countColorQuot I
```

**lemma countColorQuot\_contrIndexList\_eq\_of\_countId\_eq**[\[source\]](#)

```
lemma countColorQuot_contrIndexList_eq_of_countId_eq
  (h1 : l.contrIndexList.countId I = l.countId I) :
  l.contrIndexList.countColorQuot I = l.countColorQuot I
```

**def countSelf**[\[source\]](#)

*The number of times an index I appears in an index list.*

```
def countSelf (I : Index C.Color) : ℕ
```

**lemma countSelf\_eq\_filter\_id\_countP**[\[source\]](#)

```
lemma countSelf_eq_filter_id_countP : l.countSelf I =
  (l.val.filter (fun J => I.id = J.id)).countP (fun J => I.toColor = J.toColor)
```

**lemma countSelf\_eq\_filter\_color\_countP**[\[source\]](#)

```
lemma countSelf_eq_filter_color_countP :
  l.countSelf I =
  (l.val.filter (fun J => I.toColor = J.toColor)).countP (fun J => I.id = J.id)
```

**lemma countSelf\_of\_countId\_zero**[\[source\]](#)

```
lemma countSelf_of_countId_zero {I : Index C.Color} (h : l.countId I = 0) :
  l.countSelf I = 0
```

**lemma countSelf\_count**[\[source\]](#)

```
lemma countSelf_count (I : Index C.Color) : l.countSelf I = l.val.count I
```

**lemma countSelf\_eq\_zero**[\[source\]](#)

```
lemma countSelf_eq_zero (I : Index C.Color) : l.countSelf I = 0 ↔ I ∉ l.val
```

**lemma countSelf\_neq\_zero**[\[source\]](#)

```
lemma countSelf_neq_zero (I : Index C.Color) : l.countSelf I ≠ 0 ↔ I ∈ l.val
```

**lemma countSelf\_append**[\[source\]](#)

```
@[simp]
lemma countSelf_append (I : Index C.Color) :
  (l ++ l2).countSelf I = countSelf l I + countSelf l2 I
```

**lemma countSelf\_le\_countId**[\[source\]](#)

```
lemma countSelf_le_countId (I : Index C.Color) :
  l.countSelf I ≤ l.countId I
```

**lemma countSelf\_eq\_one\_of\_countId\_eq\_one**[\[source\]](#)

```
lemma countSelf_eq_one_of_countId_eq_one (I : Index C.Color) (h1 : l.countId I = 1)
  (hme : I ∈ l.val) : l.countSelf I = 1
```

**lemma countSelf\_contrIndexList\_le\_one**[\[source\]](#)

```
lemma countSelf_contrIndexList_le_one (I : Index C.Color) :
  l.contrIndexList.countSelf I ≤ 1
```

**lemma countSelf\_contrIndexList\_eq\_zero\_or\_one**[\[source\]](#)

```
lemma countSelf_contrIndexList_eq_zero_or_one (I : Index C.Color) :
  l.contrIndexList.countSelf I = 0 ∨ l.contrIndexList.countSelf I = 1
```

**lemma countSelf\_contrIndexList\_eq\_zero\_of\_zero**[\[source\]](#)

```
lemma countSelf_contrIndexList_eq_zero_of_zero (I : Index C.Color) (h : l.countSelf I = 0) :
  l.contrIndexList.countSelf I = 0
```

**lemma countSelf\_contrIndexList\_le\_countSelf**[\[source\]](#)

```
lemma countSelf_contrIndexList_le_countSelf (I : Index C.Color) :
  l.contrIndexList.countSelf I ≤ l.countSelf I
```

**lemma countSelf\_contrIndexList\_eq\_of\_countId\_eq**[\[source\]](#)

```
lemma countSelf_contrIndexList_eq_of_countId_eq
  (h1 : l.contrIndexList.countId I = l.countId I) :
  l.contrIndexList.countSelf I = l.countSelf I
```

**lemma countSelf\_contrIndexList\_get**[\[source\]](#)

```
@[simp]
lemma countSelf_contrIndexList_get (i : Fin l.contrIndexList.length) :
  l.contrIndexList.countSelf l.contrIndexList.val[Fin.val i] = 1
```

**def countDual**[\[source\]](#)

*The number of times the dual of an index I appears in an index list.*

```
def countDual (I : Index C.Color) : ℕ
```

**lemma countDual\_eq\_countSelf\_Dual**[\[source\]](#)

```
lemma countDual_eq_countSelf_Dual (I : Index C.Color) : l.countDual I = l.countSelf I.dual
```

**lemma countDual\_eq\_filter\_id\_countP**[\[source\]](#)

```
lemma countDual_eq_filter_id_countP : l.countDual I =
  (l.val.filter (fun J => I.id = J.id)).countP (fun J => I.toColor = C.τ (J.toColor))
```

**lemma countDual\_of\_countId\_zero**[\[source\]](#)

```
lemma countDual_of_countId_zero {I : Index C.Color} (h : l.countId I = 0) :
  l.countDual I = 0
```

**lemma countDual\_append**[\[source\]](#)

```
@[simp]
lemma countDual_append (I : Index C.Color) :
  (l ++ l2).countDual I = countDual l I + countDual l2 I
```

**lemma countDual\_contrIndexList\_eq\_of\_countId\_eq**[\[source\]](#)

```
lemma countDual_contrIndexList_eq_of_countId_eq
  (h1 : l.contrIndexList.countId I = l.countId I) :
  l.contrIndexList.countDual I = l.countDual I
```

**lemma countSelf\_eq\_countDual\_iff\_of\_isSome**[\[source\]](#)

```
lemma countSelf_eq_countDual_iff_of_isSome (hl : l.OnlyUniqueDuals)
  (i : Fin l.length) (hi : (l.getDual? i).isSome) :
  l.countSelf (l.val.get i) = l.countDual (l.val.get i) ↔
  l.colorMap i = C.τ (l.colorMap ((l.getDual? i).get hi))
  v (l.colorMap i) = C.τ (l.colorMap i)
```

**def ColorCond**[\[source\]](#)

*The condition an index and its' dual, when it exists, have dual colors.*

```
def ColorCond : Prop
```

**lemma iff\_withDual**[\[source\]](#)

```

lemma iff_withDual :
  l.ColorCond ↔ ∀ (i : l.withDual), C.τ
  (l.colorMap ((l.getDual? i).get (l.withDual_isSome i))) = l.colorMap i

```

**lemma iff\_on\_isSome**[\[source\]](#)

```

lemma iff_on_isSome : l.ColorCond ↔ ∀ (i : Fin l.length) (h : (l.getDual? i).isSome), C.τ
  (l.colorMap ((l.getDual? i).get h)) = l.colorMap i

```

**abbrev countColorCond**[\[source\]](#)

*A condition on an index list  $l$  and an index  $I$ . If the id of  $I$  appears twice in  $l$  (and  $I$  at least once) then this condition is equivalent to the dual of  $I$  having dual color to  $I$ , but written totally in terms of lists.*

```

@[simp]
abbrev countColorCond (l : IndexList C.Color) (I : Index C.Color) : Prop

```

**lemma countColorCond\_of\_filter\_eq**[\[source\]](#)

```

lemma countColorCond_of_filter_eq (l l2 : IndexList C.Color) {I : Index C.Color}
  (hf : l.val.filter (fun J => I.id = J.id) = l2.val.filter (fun J => I.id = J.id))
  (h1 : countColorCond l I) : countColorCond l2 I

```

**lemma iff\_countColorCond\_isSome**[\[source\]](#)

```

lemma iff_countColorCond_isSome (hl : l.OnlyUniqueDuals) : l.ColorCond ↔
  ∀ (i : Fin l.length) (h : (l.getDual? i).isSome), countColorCond l (l.val.get i)

```

**lemma iff\_countColorCond\_index**[\[source\]](#)

```

lemma iff_countColorCond_index (hl : l.OnlyUniqueDuals) :
  l.ColorCond ↔ ∀ (i : Fin l.length), l.countId (l.val.get i) = 2
  → countColorCond l (l.val.get i)

```

**lemma iff\_countColorCond\_mem**[\[source\]](#)

```

lemma iff_countColorCond_mem (hl : l.OnlyUniqueDuals) :
  l.ColorCond ↔ ∀ (I : Index C.Color) (h : I ∈ l.val),
  l.countId I = 2 → countColorCond l I

```

**lemma mem\_iff\_dual\_mem**[\[source\]](#)

```

lemma mem_iff_dual_mem (hl : l.OnlyUniqueDuals) (hc : l.ColorCond) (I : Index C.Color)
  (hId : l.countId I = 2) : I ∈ l.val ↔ I.dual ∈ l.val

```

**lemma iff\_countColorCond**[\[source\]](#)

```

lemma iff_countColorCond (hl : l.OnlyUniqueDuals) :
  l.ColorCond ↔ ∀ I, l.countSelf I ≠ 0 → l.countId I = 2 → countColorCond l I

```

**lemma assoc**[\[source\]](#)

```
lemma assoc (h : ColorCond (l ++ l2 ++ l3)) : ColorCond (l ++ (l2 ++ l3))
```

**lemma symm**[\[source\]](#)

```
lemma symm (hl : (l ++ l2).OnlyUniqueDuals) (h : ColorCond (l ++ l2)) :
  ColorCond (l2 ++ l)
```

**lemma inl**[\[source\]](#)

```
lemma inl (hl : (l ++ l2).OnlyUniqueDuals) (h : ColorCond (l ++ l2)) : ColorCond l
```

**lemma inr**[\[source\]](#)

```
lemma inr (hl : (l ++ l2).OnlyUniqueDuals) (h : ColorCond (l ++ l2)) : ColorCond l2
```

**lemma swap**[\[source\]](#)

```
lemma swap (hl : (l ++ l2 ++ l3).OnlyUniqueDuals) (h : ColorCond (l ++ l2 ++ l3)) :
  ColorCond (l2 ++ l ++ l3)
```

**lemma contrIndexList**[\[source\]](#)

```
lemma contrIndexList : ColorCond l.contrIndexList
```

**lemma contrIndexList\_left**[\[source\]](#)

```
lemma contrIndexList_left (hl : (l ++ l2).OnlyUniqueDuals) (h1 : (l ++ l2).ColorCond) :
  ColorCond (l.contrIndexList ++ l2)
```

**def bool**[\[source\]](#)

*A bool which is true for an index list if and only if every index and its' dual, when it exists, have dual colors.*

```
def bool (l : IndexList C.Color) : Bool
```

**lemma iff\_bool**[\[source\]](#)

```
lemma iff_bool : l.ColorCond ↔ bool l
```

**2.12 HepLean.SpaceTime.LorentzTensor.IndexNotation.IndexList.Contraction**

[HepLean/SpaceTime/LorentzTensor/IndexNotation/IndexList/Contraction.lean](#) on GitHub.

**def contrIndexList**[\[source\]](#)

*The index list formed from l by selecting only those indices in l which do not have a dual.*

```
def contrIndexList : IndexList X where
```

**lemma contrIndexList\_empty**[\[source\]](#)

```
@[simp]
lemma contrIndexList_empty : ([[]] : IndexList X).contrIndexList = { val := [] }
```

**lemma contrIndexList\_val**[\[source\]](#)

```
lemma contrIndexList_val : l.contrIndexList.val =
  l.val.filter (fun I => l.countId I = 1)
```

**def contrIndexList'**[\[source\]](#)

*An alternative form of the contracted index list.*

```
@[simp]
def contrIndexList' : IndexList X where
```

**lemma withoutDual\_sort\_eq\_filter**[\[source\]](#)

```
lemma withoutDual_sort_eq_filter : l.withoutDual.sort (fun i j => i ≤ j) =
  (List.finRange l.length).filter (fun i => i ∈ l.withoutDual)
```

**lemma contrIndexList\_eq\_contrIndexList'**[\[source\]](#)

```
lemma contrIndexList_eq_contrIndexList' : l.contrIndexList = l.contrIndexList'
```

**lemma contrIndexList\_length**[\[source\]](#)

```
@[simp]
lemma contrIndexList_length : l.contrIndexList.length = l.withoutDual.card
```

**lemma contrIndexList\_idMap**[\[source\]](#)

```
@[simp]
lemma contrIndexList_idMap (i : Fin l.contrIndexList.length) : l.contrIndexList.idMap i
  = l.idMap (l.withoutDualEquiv (Fin.cast l.contrIndexList_length i)).1
```

**lemma contrIndexList\_colorMap**[\[source\]](#)

```
@[simp]
lemma contrIndexList_colorMap (i : Fin l.contrIndexList.length) : l.contrIndexList.colorMap i
  = l.colorMap (l.withoutDualEquiv (Fin.cast l.contrIndexList_length i)).1
```

**lemma contrIndexList\_areDualInSelf**[\[source\]](#)

```
lemma contrIndexList_areDualInSelf (i j : Fin l.contrIndexList.length) :
  l.contrIndexList.AreDualInSelf i j ↔
  l.AreDualInSelf (l.withoutDualEquiv (Fin.cast l.contrIndexList_length i)).1
    (l.withoutDualEquiv (Fin.cast l.contrIndexList_length j)).1
```

**lemma contrIndexList\_getDual?**[\[source\]](#)

```
@[simp]
lemma contrIndexList_getDual? (i : Fin l.contrIndexList.length) :
  l.contrIndexList.getDual? i = none
```

**lemma contrIndexList\_withDual**[\[source\]](#)

```
@[simp]
lemma contrIndexList_withDual : l.contrIndexList.withDual = ∅
```

**lemma contrIndexList\_withUniqueDual**[\[source\]](#)

```
@[simp]
lemma contrIndexList_withUniqueDual : l.contrIndexList.withUniqueDual = ∅
```

**lemma contrIndexList\_areDualInSelf\_false**[\[source\]](#)

```
@[simp]
lemma contrIndexList_areDualInSelf_false (i j : Fin l.contrIndexList.length) :
  l.contrIndexList.AreDualInSelf i j ↔ False
```

**lemma contrIndexList\_of\_withDual\_empty**[\[source\]](#)

```
@[simp]
lemma contrIndexList_of_withDual_empty (h : l.withDual = ∅) : l.contrIndexList = l
```

**lemma contrIndexList\_contrIndexList**[\[source\]](#)

```
lemma contrIndexList_contrIndexList : l.contrIndexList.contrIndexList = l.contrIndexList
```

**lemma contrIndexList\_getDualInOther?\_self**[\[source\]](#)

```
@[simp]
lemma contrIndexList_getDualInOther?_self (i : Fin l.contrIndexList.length) :
  l.contrIndexList.getDualInOther? l.contrIndexList i = some i
```

**lemma cons\_contrIndexList\_of\_countId\_eq\_zero**[\[source\]](#)

```
lemma cons_contrIndexList_of_countId_eq_zero {I : Index X}
  (h : l.countId I = 0) :
  (l.cons I).contrIndexList = l.contrIndexList.cons I
```

**lemma cons\_contrIndexList\_of\_countId\_neq\_zero**[\[source\]](#)

```
lemma cons_contrIndexList_of_countId_neq_zero {I : Index X} (h : l.countId I ≠ 0) :
  (l.cons I).contrIndexList.val = l.contrIndexList.val.filter (fun J => ¬ I.id = J.id)
```

**lemma mem\_contrIndexList\_countId**[\[source\]](#)

```
lemma mem_contrIndexList_countId {I : Index X} (h : I ∈ l.contrIndexList.val) :
  l.countId I = 1
```

**lemma mem\_contrIndexList\_filter**[\[source\]](#)

```
lemma mem_contrIndexList_filter {I : Index X} (h : I ∈ l.contrIndexList.val) :
  l.val.filter (fun J => I.id = J.id) = [I]
```

**lemma mem\_contrIndexList\_countId\_contrIndexList**[\[source\]](#)

```
lemma mem_contrIndexList_countId_contrIndexList {I : Index X} (h : I ∈ l.contrIndexList.val) :
  l.contrIndexList.countId I = 1
```

**lemma countId\_contrIndexList\_zero\_of\_countId**[\[source\]](#)

```
lemma countId_contrIndexList_zero_of_countId (I : Index X)
  (h : l.countId I = 0) : l.contrIndexList.countId I = 0
```

**lemma countId\_contrIndexList\_le\_one**[\[source\]](#)

```
lemma countId_contrIndexList_le_one (I : Index X) :
  l.contrIndexList.countId I ≤ 1
```

**lemma countId\_contrIndexList\_eq\_one\_iff\_countId\_eq\_one**[\[source\]](#)

```
lemma countId_contrIndexList_eq_one_iff_countId_eq_one (I : Index X) :
  l.contrIndexList.countId I = 1 ↔ l.countId I = 1
```

**lemma countId\_contrIndexList\_le\_countId**[\[source\]](#)

```
lemma countId_contrIndexList_le_countId (I : Index X) :
  l.contrIndexList.countId I ≤ l.countId I
```

**lemma countId\_contrIndexList\_get**[\[source\]](#)

```
@[simp]
lemma countId_contrIndexList_get (i : Fin l.contrIndexList.length) :
  l.contrIndexList.countId l.contrIndexList.val[Fin.val i] = 1
```

**lemma filter\_id\_contrIndexList\_eq\_of\_countId\_contrIndexList**[\[source\]](#)

```
lemma filter_id_contrIndexList_eq_of_countId_contrIndexList (I : Index X)
  (h : l.contrIndexList.countId I = l.countId I) :
  l.contrIndexList.val.filter (fun J => I.id = J.id) =
  l.val.filter (fun J => I.id = J.id)
```

**lemma contrIndexList\_append\_eq\_filter**[\[source\]](#)

```
lemma contrIndexList_append_eq_filter : (l ++ l2).contrIndexList.val =
  l.contrIndexList.val.filter (fun I => l2.countId I = 0) ++
  l2.contrIndexList.val.filter (fun I => l.countId I = 0)
```

**2.13 HepLean.SpaceTime.LorentzTensor.IndexNotation.IndexList.CountId**

[HepLean/SpaceTime/LorentzTensor/IndexNotation/IndexList/CountId.lean](#) on GitHub.

**def countId**[\[source\]](#)

*The number of times the id of an index I appears in a list of indices l.*

```
def countId (I : Index X) : ℕ
```

**lemma countId\_append**[\[source\]](#)

```
@[simp]
```

```
lemma countId_append (I : Index X) : (l ++ l2).countId I = l.countId I + l2.countId I
```

**lemma countId\_eq\_length\_filter**[\[source\]](#)

```
lemma countId_eq_length_filter (I : Index X) :  
  l.countId I = (l.val.filter (fun J => I.id = J.id)).length
```

**lemma countId\_index\_neq\_zero**[\[source\]](#)

```
lemma countId_index_neq_zero (i : Fin l.length) : l.countId (l.val.get i) ≠ 0
```

**lemma countId\_append\_symm**[\[source\]](#)

```
lemma countId_append_symm (I : Index X) : (l ++ l2).countId I = (l2 ++ l).countId I
```

**lemma countId\_eq\_one\_append\_mem\_right\_self\_eq\_one**[\[source\]](#)

```
lemma countId_eq_one_append_mem_right_self_eq_one {I : Index X} (hI : I ∈ l2.val)  
  (h : (l ++ l2).countId I = 1) : l2.countId I = 1
```

**lemma countId\_eq\_one\_append\_mem\_right\_other\_eq\_zero**[\[source\]](#)

```
lemma countId_eq_one_append_mem_right_other_eq_zero {I : Index X} (hI : I ∈ l2.val)  
  (h : (l ++ l2).countId I = 1) : l.countId I = 0
```

**lemma countId\_cons\_eq\_two**[\[source\]](#)

```
@[simp]
```

```
lemma countId_cons_eq_two {I : Index X} :  
  (l.cons I).countId I = 2 ↔ l.countId I = 1
```

**lemma countId\_congr**[\[source\]](#)

```
lemma countId_congr {I J : Index X} (h : I.id = J.id) : l.countId I = l.countId J
```

**lemma countId\_neq\_zero\_mem**[\[source\]](#)

```
lemma countId_neq_zero_mem (I : Index X) (h : l.countId I ≠ 0) :  
  ∃ I', I' ∈ l.val ∧ I.id = I'.id
```

**lemma countId\_mem**[\[source\]](#)

```
lemma countId_mem (I : Index X) (hI : I ∈ l.val) : l.countId I ≠ 0
```

**lemma countId\_get\_other**[\[source\]](#)

```
lemma countId_get_other (i : Fin l.length) : l2.countId (l.val.get i) =  
  (List.finRange l2.length).countP (fun j => l.AreDualInOther l2 i j)
```

**lemma filter\_finRange**[\[source\]](#)

```
lemma filter_finRange (i : Fin l.length) :  
  List.filter (fun j => i = j) (List.finRange l.length) = [i]
```

**lemma countId\_get**[\[source\]](#)

```
lemma countId_get (i : Fin l.length) : l.countId (l.val.get i) =  
  (List.finRange l.length).countP (fun j => l.AreDualInSelf i j) + 1
```

**lemma countId\_gt\_zero\_of\_mem\_withDual**[\[source\]](#)

```
lemma countId_gt_zero_of_mem_withDual (i : Fin l.length) (h : i ∈ l.withDual) :  
  1 < l.countId (l.val.get i)
```

**lemma countId\_of\_not\_mem\_withDual**[\[source\]](#)

```
lemma countId_of_not_mem_withDual (i : Fin l.length) (h : i ∉ l.withDual) :  
  l.countId (l.val.get i) = 1
```

**lemma mem\_withDual\_iff\_countId\_gt\_one**[\[source\]](#)

```
lemma mem_withDual_iff_countId_gt_one (i : Fin l.length) :  
  i ∈ l.withDual ↔ 1 < l.countId (l.val.get i)
```

**lemma countId\_neq\_zero\_of\_mem\_withDualInOther**[\[source\]](#)

```
lemma countId_neq_zero_of_mem_withDualInOther (i : Fin l.length) (h : i ∈ l.withDualInOther l2)  
  :  
  l2.countId (l.val.get i) ≠ 0
```

**lemma countId\_of\_not\_mem\_withDualInOther**[\[source\]](#)

```
lemma countId_of_not_mem_withDualInOther (i : Fin l.length) (h : i ∉ l.withDualInOther l2) :  
  l2.countId (l.val.get i) = 0
```

**lemma mem\_withDualInOther\_iff\_countId\_neq\_zero**[\[source\]](#)

```
lemma mem_withDualInOther_iff_countId_neq_zero (i : Fin l.length) :  
  i ∈ l.withDualInOther l2 ↔ l2.countId (l.val.get i) ≠ 0
```

**lemma mem\_withoutDual\_iff\_countId\_eq\_one**[\[source\]](#)

```
lemma mem_withoutDual_iff_countId_eq_one (i : Fin l.length) :
  i ∈ l.withoutDual ↔ l.countId (l.val.get i) = 1
```

**lemma countId\_eq\_two\_of\_mem\_withUniqueDual**[\[source\]](#)

```
lemma countId_eq_two_of_mem_withUniqueDual (i : Fin l.length) (h : i ∈ l.withUniqueDual) :
  l.countId (l.val.get i) = 2
```

**lemma mem\_withUniqueDual\_of\_countId\_eq\_two**[\[source\]](#)

```
lemma mem_withUniqueDual_of_countId_eq_two (i : Fin l.length)
  (h : l.countId (l.val.get i) = 2) : i ∈ l.withUniqueDual
```

**lemma mem\_withUniqueDual\_iff\_countId\_eq\_two**[\[source\]](#)

```
lemma mem_withUniqueDual_iff_countId_eq_two (i : Fin l.length) :
  i ∈ l.withUniqueDual ↔ l.countId (l.val.get i) = 2
```

**lemma countId\_eq\_one\_of\_mem\_withUniqueDualInOther**[\[source\]](#)

```
lemma countId_eq_one_of_mem_withUniqueDualInOther (i : Fin l.length)
  (h : i ∈ l.withUniqueDualInOther l2) :
  l.countId (l.val.get i) = 1 ∧ l2.countId (l.val.get i) = 1
```

**lemma mem\_withUniqueDualInOther\_of\_countId\_eq\_one**[\[source\]](#)

```
lemma mem_withUniqueDualInOther_of_countId_eq_one (i : Fin l.length)
  (h : l.countId (l.val.get i) = 1 ∧ l2.countId (l.val.get i) = 1) :
  i ∈ l.withUniqueDualInOther l2
```

**lemma mem\_withUniqueDualInOther\_iff\_countId\_eq\_one**[\[source\]](#)

```
lemma mem_withUniqueDualInOther_iff_countId_eq_one (i : Fin l.length) :
  i ∈ l.withUniqueDualInOther l2 ↔ l.countId (l.val.get i) = 1 ∧ l2.countId (l.val.get i) = 1
```

**lemma getDual?\_countId**[\[source\]](#)

```
@[simp]
lemma getDual?_countId (i : Fin l.length) (h : (l.getDual? i).isSome) :
  l.countId (l.val[Fin.val ((l.getDual? i).get h)]) = l.countId (l.val.get i)
```

**lemma getDualInOther?\_countId\_right**[\[source\]](#)

```
@[simp]
lemma getDualInOther?_countId_right (i : Fin l.length) (h : (l.getDualInOther? l2 i).isSome) :
  l2.countId (l2.val[Fin.val ((l.getDualInOther? l2 i).get h)]) = l2.countId (l.val.get i)
```

**lemma getDualInOther?\_countId\_left**[\[source\]](#)`@[simp]`

```
lemma getDualInOther?_countId_left (i : Fin l.length) (h : (l.getDualInOther? l2 i).isSome) :
  l.countId (l2.val[Fin.val ((l.getDualInOther? l2 i).get h)]) = l.countId (l.val.get i)
```

**lemma getDual?\_isSome\_of\_countId\_eq\_two**[\[source\]](#)

```
lemma getDual?_isSome_of_countId_eq_two {i : Fin l.length}
  (h : l.countId (l.val.get i) = 2) : (l.getDual? i).isSome
```

**lemma filter\_id\_of\_countId\_eq\_zero**[\[source\]](#)

```
lemma filter_id_of_countId_eq_zero {i : Fin l.length} (h1 : l.countId (l.val.get i) = 0) :
  l.val.filter (fun J => (l.val.get i).id = J.id) = []
```

**lemma filter\_id\_of\_countId\_eq\_zero'**[\[source\]](#)

```
lemma filter_id_of_countId_eq_zero' {I : Index X} (h1 : l.countId I = 0) :
  l.val.filter (fun J => I.id = J.id) = []
```

**lemma filter\_id\_of\_countId\_eq\_one**[\[source\]](#)

```
lemma filter_id_of_countId_eq_one {i : Fin l.length} (h1 : l.countId (l.val.get i) = 1) :
  l.val.filter (fun J => (l.val.get i).id = J.id) = [l.val.get i]
```

**lemma filter\_id\_of\_countId\_eq\_one\_mem**[\[source\]](#)

```
lemma filter_id_of_countId_eq_one_mem {I : Index X} (hI : I ∈ l.val) (h : l.countId I = 1) :
  l.val.filter (fun J => I.id = J.id) = [I]
```

**lemma filter\_id\_of\_countId\_eq\_two**[\[source\]](#)

```
lemma filter_id_of_countId_eq_two {i : Fin l.length}
  (h : l.countId (l.val.get i) = 2) :
  l.val.filter (fun J => (l.val.get i).id = J.id) =
  [l.val.get i, l.val.get ((l.getDual? i).get (l.getDual?_isSome_of_countId_eq_two h))] v
  l.val.filter (fun J => (l.val.get i).id = J.id) =
  [l.val.get ((l.getDual? i).get (l.getDual?_isSome_of_countId_eq_two h)), l.val.get i]
```

**2.14 HepLean.SpaceTime.LorentzTensor.IndexNotation.IndexList.Duals**

[HepLean/SpaceTime/LorentzTensor/IndexNotation/IndexList/Duals.lean](#) on GitHub.

**def AreDualInSelf**[\[source\]](#)

*Two indices are dual if they are not equivalent, but have the same id.*

```
def AreDualInSelf (i j : Fin l.length) : Prop
```

**instance Decidable (l.AreDualInSelf i j)**

[\[source\]](#)

```
instance (i j : Fin l.length) : Decidable (l.AreDualInSelf i j)
```

**def AreDualInOther**

[\[source\]](#)

*Two indices in different IndexLists are dual to one another if they have the same id.*

```
def AreDualInOther (i : Fin l.length) (j : Fin l2.length) :
  Prop
```

**instance Decidable (l.AreDualInOther l2 i j)**

[\[source\]](#)

```
instance {l : IndexList X} {l2 : IndexList X} (i : Fin l.length) (j : Fin l2.length) :
  Decidable (l.AreDualInOther l2 i j)
```

**def getDual?**

[\[source\]](#)

*Given an i, if a dual exists in the same list, outputs the first such dual, otherwise outputs none.*

```
def getDual? (i : Fin l.length) : Option (Fin l.length)
```

**def getDualInOther?**

[\[source\]](#)

*Given an i, if a dual exists in the other list, outputs the first such dual, otherwise outputs none.*

```
def getDualInOther? (i : Fin l.length) : Option (Fin l2.length)
```

**def withDual**

[\[source\]](#)

*The finite set of indices of an index list which have a dual in that index list.*

```
def withDual : Finset (Fin l.length)
```

**def withoutDual**

[\[source\]](#)

*The finite set of indices of an index list which do not have a dual.*

```
def withoutDual : Finset (Fin l.length)
```

**def withDualInOther**

[\[source\]](#)

*The finite set of indices of an index list which have a dual in another provided index list.*

```
def withDualInOther : Finset (Fin l.length)
```

**def withUniqueDual**[\[source\]](#)

*The finite set of indices of an index list which have a unique dual in that index list.*

```
def withUniqueDual : Finset (Fin l.length)
```

**instance Decidable (i < j)**[\[source\]](#)

```
instance (i j : Option (Fin l.length)) : Decidable (i < j)
```

**def withUniqueDualLT**[\[source\]](#)

*The finite set of those indices of l which have a unique dual, and for which that dual is greater-than (determined by the ordering on Fin) then the index itself.*

```
def withUniqueDualLT : Finset (Fin l.length)
```

**def withUniqueDualGT**[\[source\]](#)

*The finite set of those indices of l which have a unique dual, and for which that dual is less-than (determined by the ordering on Fin) then the index itself.*

```
def withUniqueDualGT : Finset (Fin l.length)
```

**def withUniqueDualInOther**[\[source\]](#)

*The finite set of indices of an index list which have a unique dual in another, provided, index list.*

```
def withUniqueDualInOther : Finset (Fin l.length)
```

**lemma mem\_withDual\_of\_mem\_withUniqueDual**[\[source\]](#)

```
@[simp, noint simpNF]
lemma mem_withDual_of_mem_withUniqueDual (i : Fin l.length) (h : i ∈ l.withUniqueDual) :
  i ∈ l.withDual
```

**lemma mem\_withDual\_of\_withUniqueDual**[\[source\]](#)

```
lemma mem_withDual_of_withUniqueDual (i : l.withUniqueDual) :
  i.1 ∈ l.withDual
```

**lemma not\_mem\_withDual\_of\_withUniqueDualInOther**[\[source\]](#)

```
lemma not_mem_withDual_of_withUniqueDualInOther (i : l.withUniqueDualInOther l2) :
  i.1 ∉ l.withDual
```

**lemma mem\_withDualInOther\_of\_withUniqueDualInOther**[\[source\]](#)

```
lemma mem_withDualInOther_of_withUniqueDualInOther (i : l.withUniqueDualInOther l2) :
  i.1 ∈ l.withDualInOther l2
```

**lemma withDual\_isSome**[\[source\]](#)

```
@[simp]
lemma withDual_isSome (i : l.withDual) : (l.getDual? i).isSome
```

**lemma mem\_withDual\_iff\_isSome**[\[source\]](#)

```
@[simp]
lemma mem_withDual_iff_isSome (i : Fin l.length) : i ∈ l.withDual ↔ (l.getDual? i).isSome
```

**lemma mem\_withUniqueDual\_isSome**[\[source\]](#)

```
@[simp]
lemma mem_withUniqueDual_isSome (i : Fin l.length) (h : i ∈ l.withUniqueDual) :
  (l.getDual? i).isSome
```

**lemma mem\_withInDualOther\_iff\_isSome**[\[source\]](#)

```
@[simp]
lemma mem_withInDualOther_iff_isSome (i : Fin l.length) :
  i ∈ l.withDualInOther l2 ↔ (l.getDualInOther? l2 i).isSome
```

**lemma mem\_withUniqueDualInOther\_isSome**[\[source\]](#)

```
@[simp]
lemma mem_withUniqueDualInOther_isSome (i : Fin l.length) (hi : i ∈ l.withUniqueDualInOther l2) :
  (l.getDualInOther? l2 i).isSome
```

**lemma not\_mem\_withDual\_iff\_isNone**[\[source\]](#)

```
lemma not_mem_withDual_iff_isNone (i : Fin l.length) :
  i ∉ l.withDual ↔ (l.getDual? i).isNone
```

**lemma mem\_withDual\_iff\_exists**[\[source\]](#)

```
lemma mem_withDual_iff_exists : i ∈ l.withDual ↔ ∃ j, l.AreDualInSelf i j
```

**lemma mem\_withInDualOther\_iff\_exists**[\[source\]](#)

```
lemma mem_withInDualOther_iff_exists :
  i ∈ l.withDualInOther l2 ↔ ∃ (j : Fin l2.length), l.AreDualInOther l2 i j
```

**lemma withDual\_disjoint\_withoutDual**[\[source\]](#)

```
lemma withDual_disjoint_withoutDual : Disjoint l.withDual l.withoutDual
```

**lemma not\_mem\_withDual\_of\_mem\_withoutDual**[\[source\]](#)

```
lemma not_mem_withDual_of_mem_withoutDual (i : Fin l.length) (h : i ∈ l.withoutDual) :
  i ∉ l.withDual
```

**lemma withDual\_union\_withoutDual**[\[source\]](#)

```
lemma withDual_union_withoutDual : l.withDual ∪ l.withoutDual = Finset.univ
```

**lemma mem\_withUniqueDual\_of\_mem\_withUniqueDualLT**[\[source\]](#)

```
lemma mem_withUniqueDual_of_mem_withUniqueDualLT (i : Fin l.length) (h : i ∈ l.withUniqueDualLT) :
  i ∈ l.withUniqueDual
```

**lemma mem\_withUniqueDual\_of\_mem\_withUniqueDualGT**[\[source\]](#)

```
lemma mem_withUniqueDual_of_mem_withUniqueDualGT (i : Fin l.length) (h : i ∈ l.withUniqueDualGT) :
  i ∈ l.withUniqueDual
```

**lemma withUniqueDualLT\_disjoint\_withUniqueDualGT**[\[source\]](#)

```
lemma withUniqueDualLT_disjoint_withUniqueDualGT :
  Disjoint l.withUniqueDualLT l.withUniqueDualGT
```

**lemma withUniqueDualLT\_union\_withUniqueDualGT**[\[source\]](#)

```
lemma withUniqueDualLT_union_withUniqueDualGT :
  l.withUniqueDualLT ∪ l.withUniqueDualGT = l.withUniqueDual
```

**instance Decidable (l.AreDualInSelf i j)**[\[source\]](#)

```
instance (i j : Fin l.length) : Decidable (l.AreDualInSelf i j)
```

**lemma AreDualInSelf.symm**[\[source\]](#)

```
@[symm]
lemma symm (h : l.AreDualInSelf i j) : l.AreDualInSelf j i
```

**lemma self\_false**[\[source\]](#)

```
@[simp]
lemma self_false (i : Fin l.length) : ¬ l.AreDualInSelf i i
```

**lemma append\_inl\_inl**[\[source\]](#)

```
@[simp]
lemma append_inl_inl : (l ++ l2).AreDualInSelf (appendEquiv (Sum.inl i)) (appendEquiv (Sum.inl j))
  ↔ l.AreDualInSelf i j
```

**lemma append\_inr\_inr**[\[source\]](#)

```
@[simp]
lemma append_inr_inr (l l2 : IndexList X) (i j : Fin l2.length) :
  (l ++ l2).AreDualInSelf (appendEquiv (Sum.inr i)) (appendEquiv (Sum.inr j))
```

```
↔ l2.AreDualInSelf i j
```

### lemma append\_inl\_inr

[\[source\]](#)

```
@[simp]
lemma append_inl_inr (l l2 : IndexList X) (i : Fin l.length) (j : Fin l2.length) :
  (l ++ l2).AreDualInSelf (appendEquiv (Sum.inl i)) (appendEquiv (Sum.inr j)) =
  l.AreDualInOther l2 i j
```

### lemma append\_inr\_inl

[\[source\]](#)

```
@[simp]
lemma append_inr_inl (l l2 : IndexList X) (i : Fin l2.length) (j : Fin l.length) :
  (l ++ l2).AreDualInSelf (appendEquiv (Sum.inr i)) (appendEquiv (Sum.inl j)) =
  l2.AreDualInOther l i j
```

### instance Decidable (l.AreDualInOther l2 i j)

[\[source\]](#)

```
instance {l : IndexList X} {l2 : IndexList X} (i : Fin l.length) (j : Fin l2.length) :
  Decidable (l.AreDualInOther l2 i j)
```

### lemma AreDualInOther.symm

[\[source\]](#)

```
@[symm]
lemma symm (h : l.AreDualInOther l2 i j) : l2.AreDualInOther l j i
```

### lemma append\_of\_inl

[\[source\]](#)

```
@[simp]
lemma append_of_inl (i : Fin l.length) (j : Fin l3.length) :
  (l ++ l2).AreDualInOther l3 (appendEquiv (Sum.inl i)) j ↔ l.AreDualInOther l3 i j
```

### lemma append\_of\_inr

[\[source\]](#)

```
@[simp]
lemma append_of_inr (i : Fin l2.length) (j : Fin l3.length) :
  (l ++ l2).AreDualInOther l3 (appendEquiv (Sum.inr i)) j ↔ l2.AreDualInOther l3 i j
```

### lemma of\_append\_inl

[\[source\]](#)

```
@[simp]
lemma of_append_inl (i : Fin l.length) (j : Fin l2.length) :
  l.AreDualInOther (l2 ++ l3) i (appendEquiv (Sum.inl j)) ↔ l.AreDualInOther l2 i j
```

### lemma of\_append\_inr

[\[source\]](#)

```
@[simp]
lemma of_append_inr (i : Fin l.length) (j : Fin l3.length) :
  l.AreDualInOther (l2 ++ l3) i (appendEquiv (Sum.inr j)) ↔ l.AreDualInOther l3 i j
```

**lemma getDual?\_isSome\_iff\_exists**[\[source\]](#)

```
lemma getDual?_isSome_iff_exists (i : Fin l.length) :
  (l.getDual? i).isSome ↔ ∃ j, l.AreDualInSelf i j
```

**lemma getDual?\_of\_areDualInSelf**[\[source\]](#)

```
lemma getDual?_of_areDualInSelf (h : l.AreDualInSelf i j) :
  l.getDual? j = i ∨ l.getDual? i = j ∨ l.getDual? j = l.getDual? i
```

**lemma getDual?\_neq\_self**[\[source\]](#)

```
@[simp]
lemma getDual?_neq_self (i : Fin l.length) : ¬ l.getDual? i = some i
```

**lemma getDual?\_get\_neq\_self**[\[source\]](#)

```
@[simp]
lemma getDual?_get_neq_self (i : Fin l.length) (h : (l.getDual? i).isSome) :
  ¬ (l.getDual? i).get h = i
```

**lemma getDual?\_get\_id**[\[source\]](#)

```
@[simp]
lemma getDual?_get_id (i : Fin l.length) (h : (l.getDual? i).isSome) :
  l.idMap ((l.getDual? i).get h) = l.idMap i
```

**lemma getDual?\_get\_areDualInSelf**[\[source\]](#)

```
@[simp]
lemma getDual?_get_areDualInSelf (i : Fin l.length) (h : (l.getDual? i).isSome) :
  l.AreDualInSelf ((l.getDual? i).get h) i
```

**lemma getDual?\_areDualInSelf\_get**[\[source\]](#)

```
@[simp]
lemma getDual?_areDualInSelf_get (i : Fin l.length) (h : (l.getDual? i).isSome) :
  l.AreDualInSelf i ((l.getDual? i).get h)
```

**lemma getDual?\_getDual?\_get\_isSome**[\[source\]](#)

```
@[simp]
lemma getDual?_getDual?_get_isSome (i : Fin l.length) (h : (l.getDual? i).isSome) :
  (l.getDual? ((l.getDual? i).get h)).isSome
```

**lemma getDualInOther?\_isSome\_iff\_exists**[\[source\]](#)

```
lemma getDualInOther?_isSome_iff_exists (i : Fin l.length) :
  (l.getDualInOther? l2 i).isSome ↔ ∃ j, l.AreDualInOther l2 i j
```

**lemma getDualInOther?\_id**[\[source\]](#)

```
@[simp]
lemma getDualInOther?_id (i : Fin l.length) (h : (l.getDualInOther? l2 i).isSome) :
  l2.idMap ((l.getDualInOther? l2 i).get h) = l.idMap i
```

**lemma getDualInOther?\_get\_areDualInOther**[\[source\]](#)

```
@[simp]
lemma getDualInOther?_get_areDualInOther (i : Fin l.length) (h : (l.getDualInOther? l2 i).
  isSome) :
  l2.AreDualInOther l ((l.getDualInOther? l2 i).get h) i
```

**lemma getDualInOther?\_areDualInOther\_get**[\[source\]](#)

```
@[simp]
lemma getDualInOther?_areDualInOther_get (i : Fin l.length) (h : (l.getDualInOther? l2 i).
  isSome) :
  l.AreDualInOther l2 i ((l.getDualInOther? l2 i).get h)
```

**lemma getDualInOther?\_getDualInOther?\_get\_isSome**[\[source\]](#)

```
@[simp]
lemma getDualInOther?_getDualInOther?_get_isSome (i : Fin l.length)
  (h : (l.getDualInOther? l2 i).isSome) :
  (l2.getDualInOther? l ((l.getDualInOther? l2 i).get h)).isSome
```

**lemma getDualInOther?\_self\_isSome**[\[source\]](#)

```
@[simp]
lemma getDualInOther?_self_isSome (i : Fin l.length) :
  (l.getDualInOther? l i).isSome
```

**lemma getDualInOther?\_getDualInOther?\_get\_self**[\[source\]](#)

```
@[simp]
lemma getDualInOther?_getDualInOther?_get_self (i : Fin l.length) :
  l.getDualInOther? l ((l.getDualInOther? l i).get (getDualInOther?_self_isSome l i)) =
  some ((l.getDualInOther? l i).get (getDualInOther?_self_isSome l i))
```

**2.15 HepLean.SpaceTime.LorentzTensor.IndexNotation.IndexList.Equivs**

[HepLean/SpaceTime/LorentzTensor/IndexNotation/IndexList/Equivs.lean](#) on GitHub.

**def withoutDualEquiv**[\[source\]](#)

*An equivalence from  $\text{Fin } l.\text{withoutDual}.\text{card}$  to  $l.\text{withoutDual}$  determined by the order on  $l.\text{withoutDual}$  inherited from  $\text{Fin}$ .*

```
def withoutDualEquiv : Fin l.withoutDual.card ≃ l.withoutDual
```

**lemma list\_ofFn\_withoutDualEquiv\_eq\_sort**[\[source\]](#)

```
lemma list_ofFn_withoutDualEquiv_eq_sort :
  List.ofFn (Subtype.val ∘ l.withoutDualEquiv) = l.withoutDual.sort (fun i j => i ≤ j)
```

**def dualEquiv**[\[source\]](#)

*An equivalence splitting the indices of an index list  $l$  into those indices which have a dual in  $l$  and those which do not have a dual in  $l$ .*

```
def dualEquiv : l.withDual @ Fin l.withoutDual.card ≈ Fin l.length
```

**def getDualEquiv**[\[source\]](#)

*An equivalence from  $l.withUniqueDual$  to itself obtained by taking the dual index.*

```
def getDualEquiv : l.withUniqueDual ≈ l.withUniqueDual where
```

**lemma getDual?\_getDualEquiv**[\[source\]](#)

```
@[simp]
lemma getDual?_getDualEquiv (i : l.withUniqueDual) : l.getDual? (l.getDualEquiv i) = i
```

**def getDualInOtherEquiv**[\[source\]](#)

*An equivalence from  $l.withUniqueDualInOther\ l2$  to  $l2.withUniqueDualInOther\ l$  obtained by taking the dual index.*

```
def getDualInOtherEquiv : l.withUniqueDualInOther l2 ≈ l2.withUniqueDualInOther l where
```

**lemma getDualInOtherEquiv\_self\_refl**[\[source\]](#)

```
@[simp]
lemma getDualInOtherEquiv_self_refl : l.getDualInOtherEquiv l = Equiv.refl _
```

**lemma getDualInOtherEquiv\_symm**[\[source\]](#)

```
@[simp]
lemma getDualInOtherEquiv_symm : (l.getDualInOtherEquiv l2).symm = l2.getDualInOtherEquiv l
```

**def withUniqueDualCastLeft**[\[source\]](#)

*An equivalence casting  $withUniqueDualInOther$  based on an equality of the left index list.*

```
def withUniqueDualCastLeft (h : l = l3) :
  l.withUniqueDualInOther l2 ≈ l3.withUniqueDualInOther l2 where
```

**def withUniqueDualCastRight**[\[source\]](#)

*An equivalence casting  $withUniqueDualInOther$  based on an equality of the right index list.*

```
def withUniqueDualCastRight (h : l2 = l3) :
  l.withUniqueDualInOther l2 ≈ l.withUniqueDualInOther l3 where
```

**def withUniqueDualCast**

[\[source\]](#)

*An equivalence casting withUniqueDualInOther based on an equality of both index lists.*

```
def withUniqueDualCast {l1 l2 l1' l2' : IndexList X} (h : l1 = l1') (h2 : l2 = l2') :
  l1.withUniqueDualInOther l2 ≈ l1'.withUniqueDualInOther l2' where
```

**lemma getDualInOtherEquiv\_cast\_left**

[\[source\]](#)

```
lemma getDualInOtherEquiv_cast_left (h : l = l3) :
  l.getDualInOtherEquiv l2 = ((withUniqueDualCastLeft l l2 l3 h).trans
    (l3.getDualInOtherEquiv l2)).trans (withUniqueDualCastRight l2 l l3 h).symm
```

**lemma getDualInOtherEquiv\_cast\_right**

[\[source\]](#)

```
lemma getDualInOtherEquiv_cast_right (h : l2 = l3) :
  l.getDualInOtherEquiv l2 = ((withUniqueDualCastRight l l2 l3 h).trans
    (l.getDualInOtherEquiv l3)).trans (withUniqueDualCastLeft l2 l l3 h).symm
```

**lemma getDualInOtherEquiv\_cast**

[\[source\]](#)

```
lemma getDualInOtherEquiv_cast {l1 l2 l1' l2' : IndexList X} (h : l1 = l1') (h2 : l2 = l2') :
  l1.getDualInOtherEquiv l2 = ((withUniqueDualCast h h2).trans
    (l1'.getDualInOtherEquiv l2')).trans (withUniqueDualCast h2 h).symm
```

**lemma option\_not\_lt**

[\[source\]](#)

```
lemma option_not_lt (i j : Option (Fin l.length)) : i < j → i ≠ j → ¬ j < i
```

**lemma lt\_option\_of\_not**

[\[source\]](#)

```
lemma lt_option_of_not (i j : Option (Fin l.length)) : ¬ j < i → i ≠ j → i < j
```

**def withUniqueDualLTEquivGT**

[\[source\]](#)

*The equivalence between l.withUniqueDualLT and l.withUniqueDualGT obtained by taking an index to its dual.*

```
def withUniqueDualLTEquivGT : l.withUniqueDualLT ≈ l.withUniqueDualGT where
```

**def withUniqueLTGTEquiv**

[\[source\]](#)

*An equivalence from l.withUniqueDualLT ⊗ l.withUniqueDualLT to l.withUniqueDual. The first l.withUniqueDualLT are taken to themselves, whilst the second copy are taken to their dual.*

```
def withUniqueLTGTEquiv : l.withUniqueDualLT ⊗ l.withUniqueDualLT ≈ l.withUniqueDual
```

## 2.16 HepLean.SpaceTime.LorentzTensor.IndexNotation.IndexList.Normalize

[HepLean/SpaceTime/LorentzTensor/IndexNotation/IndexList/Normalize.lean](#) on GitHub.

### lemma dedup\_map\_of\_injective'

[\[source\]](#)

```
lemma dedup_map_of_injective' {α : Type} [DecidableEq α] (f : α → ℕ)
  (l : List α) (hf : ∀ I ∈ l, ∀ J ∈ l, f I = f J ↔ I = J) :
  (l.map f).dedup = l.dedup.map f
```

### lemma index0f\_map

[\[source\]](#)

```
lemma index0f_map {α : Type} [DecidableEq α]
  (f : α → ℕ) (l : List α) (n : α) (hf : ∀ I ∈ l, ∀ J, f I = f J ↔ I = J) :
  l.index0f n = (l.map f).index0f (f n)
```

### lemma index0f\_map\_fin

[\[source\]](#)

```
lemma index0f_map_fin {α : Type} {m : ℕ} [DecidableEq α]
  (f : α → (Fin m)) (l : List α) (n : α) (hf : ∀ I ∈ l, ∀ J, f I = f J ↔ I = J) :
  l.index0f n = (l.map f).index0f (f n)
```

### lemma index0f\_map'

[\[source\]](#)

```
lemma index0f_map' {α : Type} [DecidableEq α]
  (f : α → ℕ) (g : α → α) (l : List α) (n : α)
  (hf : ∀ I ∈ l, ∀ J, f I = f J ↔ g I = g J)
  (hg : ∀ I ∈ l, g I = I)
  (hs : ∀ I, f I = f (g I)) :
  l.index0f (g n) = (l.map f).index0f (f n)
```

### lemma filter\_dedup

[\[source\]](#)

```
lemma filter_dedup {α : Type} [DecidableEq α] (l : List α) (p : α → Prop) [DecidablePred p] :
  (l.filter p).dedup = (l.dedup.filter p)
```

### lemma findIdx?\_map

[\[source\]](#)

```
lemma findIdx?_map {α β : Type} (p : α → Bool)
  (f : β → α) (l : List β) :
  List.findIdx? p (List.map f l) = List.findIdx? (p ∘ f) l
```

### lemma findIdx?\_on\_finRange

[\[source\]](#)

```
lemma findIdx?_on_finRange {n : ℕ} (p : Fin n.succ → Prop) [DecidablePred p]
  (hp : ¬ p 0) :
  List.findIdx? p (List.finRange n.succ) =
  Option.map (fun i => i + 1) (List.findIdx? (p ∘ Fin.succ) (List.finRange n))
```

### lemma findIdx?\_on\_finRange\_eq\_findIdx

[\[source\]](#)

```
lemma findIdx?_on_finRange_eq_findIdx {n : ℕ} (p : Fin n → Prop) [DecidablePred p] :
  List.findIdx? p (List.finRange n) = Option.map Fin.val (Fin.find p)
```

**def idListFin**[\[source\]](#)

The list of elements of  $\text{Fin } l.\text{length } i$  is in  $\text{idListFin}$  if  $l.\text{idMap } i$  appears for the first time in the  $i$ th spot.

For example, for the list  $['u_1', 'u_2', 'u_1']$  the  $\text{idListFin}$  is  $[0, 1]$ .

```
def idListFin : List (Fin l.length)
```

**lemma idListFin\_getDualInOther?**[\[source\]](#)

```
lemma idListFin_getDualInOther? : l.idListFin =  
  (List.finRange l.length).filter (fun i => l.getDualInOther? l i = i)
```

**def idList**[\[source\]](#)

The list of ids keeping only the first appearance of each id.

```
def idList : List ℕ
```

**lemma idList\_getDualInOther?**[\[source\]](#)

```
lemma idList_getDualInOther? : l.idList =  
  List.map l.idMap ((List.finRange l.length).filter (fun i => l.getDualInOther? l i = i))
```

**lemma mem\_idList\_of\_mem**[\[source\]](#)

```
lemma mem_idList_of_mem {I : Index X} (hI : I ∈ l.val) : I.id ∈ l.idList
```

**lemma idMap\_mem\_idList**[\[source\]](#)

```
lemma idMap_mem_idList (i : Fin l.length) : l.idMap i ∈ l.idList
```

**lemma idList\_indexOf\_mem**[\[source\]](#)

```
@[simp]  
lemma idList_indexOf_mem {I J : Index X} (hI : I ∈ l.val) (hJ : J ∈ l.val) :  
  l.idList.indexOf I.id = l.idList.indexOf J.id ↔ I.id = J.id
```

**lemma idList\_indexOf\_get**[\[source\]](#)

```
lemma idList_indexOf_get (i : Fin l.length) :  
  l.idList.indexOf (l.idMap i) = l.idListFin.indexOf ((l.getDualInOther? l i).get  
    (getDualInOther?_self_isSome l i))
```

**def GetDualCast**[\[source\]](#)

Two  $\text{IndexLists}$  are related by  $\text{GetDualCast}$  if they are the same length, and have the same dual structure.

For example,  $['u_1', 'u_2', 'u_1']$  and  $['u_4', 'u_5', 'u_4']$  are related by  $\text{GetDualCast}$ , but  $['u_1', 'u_2', 'u_1']$  and  $['u_1', 'u_2', 'u_3']$  are not.

```
def GetDualCast : Prop
```

**lemma GetDualCast.symm**

[\[source\]](#)

```
lemma symm (h : GetDualCast l l2) : GetDualCast l2 l
```

**lemma getDual?\_isSome\_iff**

[\[source\]](#)

```
lemma getDual?_isSome_iff (h : GetDualCast l l2) (i : Fin l.length) :
  (l.getDual? i).isSome ↔ (l2.getDual? (Fin.cast h.1 i)).isSome
```

**lemma getDual?\_get**

[\[source\]](#)

```
lemma getDual?_get (h : GetDualCast l l2) (i : Fin l.length) (h1 : (l.getDual? i).isSome) :
  (l.getDual? i).get h1 = Fin.cast h.1.symm ((l2.getDual? (Fin.cast h.1 i)).get
  ((getDual?_isSome_iff h i).mp h1))
```

**lemma areDualInSelf\_of**

[\[source\]](#)

```
lemma areDualInSelf_of (h : GetDualCast l l2) (i j : Fin l.length) (hA : l.AreDualInSelf i j) :
  l2.AreDualInSelf (Fin.cast h.1 i) (Fin.cast h.1 j)
```

**lemma areDualInSelf\_iff**

[\[source\]](#)

```
lemma areDualInSelf_iff (h : GetDualCast l l2) (i j : Fin l.length) : l.AreDualInSelf i j ↔
  l2.AreDualInSelf (Fin.cast h.1 i) (Fin.cast h.1 j)
```

**lemma idMap\_eq\_of**

[\[source\]](#)

```
lemma idMap_eq_of (h : GetDualCast l l2) (i j : Fin l.length) (hm : l.idMap i = l.idMap j) :
  l2.idMap (Fin.cast h.1 i) = l2.idMap (Fin.cast h.1 j)
```

**lemma idMap\_eq\_iff**

[\[source\]](#)

```
lemma idMap_eq_iff (h : GetDualCast l l2) (i j : Fin l.length) :
  l.idMap i = l.idMap j ↔ l2.idMap (Fin.cast h.1 i) = l2.idMap (Fin.cast h.1 j)
```

**lemma iff\_idMap\_eq**

[\[source\]](#)

```
lemma iff_idMap_eq : GetDualCast l l2 ↔
  l.length = l2.length ∧ ∀ (h : l.length = l2.length) i j,
  l.idMap i = l.idMap j ↔ l2.idMap (Fin.cast h i) = l2.idMap (Fin.cast h j)
```

**lemma GetDualCast.refl**

[\[source\]](#)

```
lemma refl (l : IndexList X) : GetDualCast l l
```

**lemma GetDualCast.trans**[\[source\]](#)

```
@[trans]
lemma trans (h : GetDualCast l l2) (h' : GetDualCast l2 l3) : GetDualCast l l3
```

**lemma getDualInOther?\_get**[\[source\]](#)

```
lemma getDualInOther?_get (h : GetDualCast l l2) (i : Fin l.length) :
  (l.getDualInOther? l i).get (getDualInOther?_self_isSome l i) = (Fin.cast h.l.symm)
  ((l2.getDualInOther? l2 (Fin.cast h.l i)).get
   (getDualInOther?_self_isSome l2 (Fin.cast h.left i)))
```

**lemma countId\_cast**[\[source\]](#)

```
lemma countId_cast (h : GetDualCast l l2) (i : Fin l.length) :
  l.countId (l.val.get i) = l2.countId (l2.val.get (Fin.cast h.l i))
```

**lemma idListFin\_cast**[\[source\]](#)

```
lemma idListFin_cast (h : GetDualCast l l2) :
  List.map (Fin.cast h.l) l.idListFin = l2.idListFin
```

**lemma idList\_indexOf**[\[source\]](#)

```
lemma idList_indexOf (h : GetDualCast l l2) (i : Fin l.length) :
  l2.idList.indexOf (l2.idMap (Fin.cast h.l i)) = l.idList.indexOf (l.idMap i)
```

**def normalize**[\[source\]](#)

*Given an index list  $l$ , the corresponding index list where id's are set to sequentially lowest values.*

*E.g. on  $['u_1', 'u_3', 'u_1']$  this gives  $['u_0', 'u_1', 'u_0']$ .*

```
def normalize : IndexList X where
```

**lemma normalize\_eq\_map**[\[source\]](#)

```
lemma normalize_eq_map :
  l.normalize.val = List.map (fun I => ⟨I.toColor, l.idList.indexOf I.id⟩) l.val
```

**lemma normalize\_length**[\[source\]](#)

```
@[simp]
lemma normalize_length : l.normalize.length = l.length
```

**lemma normalize\_val\_length**[\[source\]](#)

```
@[simp]
lemma normalize_val_length : l.normalize.val.length = l.val.length
```

**lemma normalize\_colorMap**[\[source\]](#)

```
@[simp]
lemma normalize_colorMap : l.normalize.colorMap = l.colorMap ◦ Fin.cast l.normalize_length
```

**lemma colorMap\_normalize**[\[source\]](#)

```
lemma colorMap_normalize :
  l.colorMap = l.normalize.colorMap ◦ Fin.cast l.normalize_length.symm
```

**lemma normalize\_countId**[\[source\]](#)

```
@[simp]
lemma normalize_countId (i : Fin l.normalize.length) :
  l.normalize.countId l.normalize.val[Fin.val i] =
  l.countId (l.val.get (i, by simp using i.2))
```

**lemma normalize\_countId'**[\[source\]](#)

```
@[simp]
lemma normalize_countId' (i : Fin l.length) (hi : i.1 < l.normalize.length) :
  l.normalize.countId (l.normalize.val[Fin.val i]) = l.countId (l.val.get i)
```

**lemma normalize\_countId\_mem**[\[source\]](#)

```
@[simp]
lemma normalize_countId_mem (I : Index X) (h : I ∈ l.val) :
  l.normalize.countId (I.toColor, List.indexOf I.id l.idList) = l.countId I
```

**lemma normalize\_filter\_countId\_eq\_one**[\[source\]](#)

```
lemma normalize_filter_countId_eq_one :
  List.map Index.id (l.normalize.val.filter (fun I => l.normalize.countId I = 1))
  = List.map l.idList.indexOf (List.map Index.id (l.val.filter (fun I => l.countId I = 1)))
```

**lemma normalize\_idMap\_apply**[\[source\]](#)

```
@[simp]
lemma normalize_idMap_apply (i : Fin l.normalize.length) :
  l.normalize.idMap i = l.idList.indexOf (l.val.get (i, by simp using i.2)).id
```

**lemma normalize\_getDualCast\_self**[\[source\]](#)

```
@[simp]
lemma normalize_getDualCast_self : GetDualCast l.normalize l
```

**lemma self\_getDualCast\_normalize**[\[source\]](#)

```
@[simp]
lemma self_getDualCast_normalize : GetDualCast l l.normalize
```

**lemma normalize\_filter\_countId\_not\_eq\_one**[\[source\]](#)

```
lemma normalize_filter_countId_not_eq_one :
  List.map Index.id (l.normalize.val.filter (fun I => ¬ l.normalize.countId I = 1))
  = List.map l.idList.indexOf
    (List.map Index.id (l.val.filter (fun I => ¬ l.countId I = 1)))
```

**lemma to\_normalize**[\[source\]](#)

```
lemma to_normalize (h : GetDualCast l l2) : GetDualCast l.normalize l2.normalize
```

**lemma normalize\_idMap\_eq**[\[source\]](#)

```
lemma normalize_idMap_eq (h : GetDualCast l l2) :
  l.normalize.idMap = l2.normalize.idMap ∘ Fin.cast h.to_normalize.1
```

**lemma GetDualCast.iff\_normalize**[\[source\]](#)

```
lemma iff_normalize : GetDualCast l l2 ↔
  l.normalize.length = l2.normalize.length ∧ ∀ (h : l.normalize.length = l2.normalize.length)
  ,
  l.normalize.idMap = l2.normalize.idMap ∘ Fin.cast h
```

**lemma normalize\_normalize**[\[source\]](#)

```
@[simp]
lemma normalize_normalize : l.normalize.normalize = l.normalize
```

**lemma normalize\_length\_eq\_of\_eq\_length**[\[source\]](#)

```
lemma normalize_length_eq_of_eq_length (h : l.length = l2.length) :
  l.normalize.length = l2.normalize.length
```

**lemma normalize\_colorMap\_eq\_of\_eq\_colorMap**[\[source\]](#)

```
lemma normalize_colorMap_eq_of_eq_colorMap (h : l.length = l2.length)
  (hc : l.colorMap = l2.colorMap ∘ Fin.cast h) :
  l.normalize.colorMap = l2.normalize.colorMap ∘
  Fin.cast (normalize_length_eq_of_eq_length l l2 h)
```

**def Reindexing**[\[source\]](#)

*Two ColorIndexList are said to be reindexes of one another if they: 1. have the same length. 2. every corresponding index has the same color, and duals which correspond.*

*Note: This does not allow for reordering of indices.*

```
def Reindexing : Prop
```

**lemma iff\_getDualCast**[\[source\]](#)

```
lemma iff_getDualCast : Reindexing l l2 ↔ GetDualCast l l2
  ∧ ∀ (h : l.length = l2.length), l.colorMap = l2.colorMap ∘ Fin.cast h
```

**lemma Reindexing.iff\_normalize**[\[source\]](#)

```
lemma iff_normalize : Reindexing l l2 ↔ l.normalize = l2.normalize
```

**lemma Reindexing.refl**[\[source\]](#)

```
lemma refl (l : IndexList X) : Reindexing l l
```

**lemma Reindexing.symm**[\[source\]](#)

```
@[symm]
lemma symm (h : Reindexing l l2) : Reindexing l2 l
```

**lemma Reindexing.trans**[\[source\]](#)

```
@[trans]
lemma trans (h : Reindexing l l2) (h' : Reindexing l2 l3) : Reindexing l l3
```

**lemma normalize\_reindexing\_self**[\[source\]](#)

```
@[simp]
lemma normalize_reindexing_self : Reindexing l l.normalize l
```

**lemma self\_reindexing\_normalize**[\[source\]](#)

```
@[simp]
lemma self_reindexing_normalize : Reindexing l l.normalize
```

**2.17 HepLean.SpaceTime.LorentzTensor.IndexNotation.IndexList.OnlyUniqueDuals**

[HepLean/SpaceTime/LorentzTensor/IndexNotation/IndexList/OnlyUniqueDuals.lean](#) on GitHub.

**def OnlyUniqueDuals**[\[source\]](#)

*The conditon on an index list which is true if and only if for every index with a dual, that dual is unique.*

```
def OnlyUniqueDuals : Prop
```

**lemma iff\_unique\_forall**[\[source\]](#)

```
lemma iff_unique_forall :
  l.OnlyUniqueDuals ↔
  ∀ (i : l.withDual) j, l.AreDualInSelf i j → j = l.getDual? i
```

**lemma iff\_countId\_leq\_two**[\[source\]](#)

```
lemma iff_countId_leq_two :
  l.OnlyUniqueDuals ↔ ∀ i, l.countId (l.val.get i) ≤ 2
```

**lemma iff\_countId\_leq\_two'**[\[source\]](#)

```
lemma iff_countId_leq_two' : l.OnlyUniqueDuals ↔ ∀ I, l.countId I ≤ 2
```

**lemma inl**[\[source\]](#)

```
lemma inl (h : (l ++ l2).OnlyUniqueDuals) : l.OnlyUniqueDuals
```

**lemma inr**[\[source\]](#)

```
lemma inr (h : (l ++ l2).OnlyUniqueDuals) : l2.OnlyUniqueDuals
```

**lemma symm'**[\[source\]](#)

```
lemma symm' (h : (l ++ l2).OnlyUniqueDuals) : (l2 ++ l).OnlyUniqueDuals
```

**lemma symm**[\[source\]](#)

```
lemma symm : (l ++ l2).OnlyUniqueDuals ↔ (l2 ++ l).OnlyUniqueDuals
```

**lemma swap**[\[source\]](#)

```
lemma swap (h : (l ++ l2 ++ l3).OnlyUniqueDuals) : (l2 ++ l ++ l3).OnlyUniqueDuals
```

**lemma contrIndexList**[\[source\]](#)

```
lemma contrIndexList (h : l.OnlyUniqueDuals) : l.contrIndexList.OnlyUniqueDuals
```

**lemma contrIndexList\_left**[\[source\]](#)

```
lemma contrIndexList_left (h1 : (l ++ l2).OnlyUniqueDuals) :
  (l.contrIndexList ++ l2).OnlyUniqueDuals
```

**lemma contrIndexList\_right**[\[source\]](#)

```
lemma contrIndexList_right (h1 : (l ++ l2).OnlyUniqueDuals) :
  (l ++ l2.contrIndexList).OnlyUniqueDuals
```

**lemma contrIndexList\_append**[\[source\]](#)

```
lemma contrIndexList_append (h1 : (l ++ l2).OnlyUniqueDuals) :
  (l.contrIndexList ++ l2.contrIndexList).OnlyUniqueDuals
```

**lemma countId\_of\_OnlyUniqueDuals**[\[source\]](#)

```
lemma countId_of_OnlyUniqueDuals (h : l.OnlyUniqueDuals) (I : Index X) :
  l.countId I ≤ 2
```

**lemma countId\_eq\_two\_ofcontrIndexList\_left\_of\_OnlyUniqueDuals**[\[source\]](#)

```
lemma countId_eq_two_ofcontrIndexList_left_of_OnlyUniqueDuals
  (h : (l ++ l2).OnlyUniqueDuals) (I : Index X) (h' : (l.contrIndexList ++ l2).countId I = 2)
  :
  (l ++ l2).countId I = 2
```

**2.18 HepLean.SpaceTime.LorentzTensor.IndexNotation.IndexList.Subperm**

[HepLean/SpaceTime/LorentzTensor/IndexNotation/IndexList/Subperm.lean](#) on GitHub.

**def Subperm**[\[source\]](#)

*An IndexList  $l$  is a subpermutation of  $l2$  if there are no duals in  $l$ , and every element of  $l$  has a unique dual in  $l2$ .*

```
def Subperm : Prop
```

**lemma iff\_countId\_fin**[\[source\]](#)

```
lemma iff_countId_fin : l.Subperm l2 ↔
  ∀ i, l.countId (l.val.get i) = 1 ∧ l2.countId (l.val.get i) = 1
```

**lemma iff\_countId**[\[source\]](#)

```
lemma iff_countId : l.Subperm l2 ↔
  ∀ I, l.countId I ≤ 1 ∧ (l.countId I = 1 → l2.countId I = 1)
```

**lemma trans**[\[source\]](#)

```
lemma trans (h1 : l.Subperm l2) (h2 : l2.Subperm l3) : l.Subperm l3
```

**lemma fst\_eq\_contrIndexList**[\[source\]](#)

```
lemma fst_eq_contrIndexList (h : l.Subperm l2) : l.contrIndexList = l
```

**lemma symm**[\[source\]](#)

```
lemma symm (h : l.Subperm l2) (h1 : l.length = l2.length) : l2.Subperm l
```

**lemma contr\_refl**[\[source\]](#)

```
lemma contr_refl : l.contrIndexList.Subperm l.contrIndexList
```

**2.19 HepLean.SpaceTime.LorentzTensor.IndexNotation.IndexString**

[HepLean/SpaceTime/LorentzTensor/IndexNotation/IndexString.lean](#) on GitHub.

**def stringToPreIndexList**[\[source\]](#)

*Takes in a list of characters and splits it into a list of characters at those characters which are notation characters e.g. 'u'.*

```
def stringToPreIndexList (l : List Char) : List (List Char)
```

**def listCharIsIndexString**[\[source\]](#)

*A bool which is true given a list of characters if and only if every element of the corresponding stringToPreIndexList correspondings to an index.*

```
def listCharIsIndexString (l : List Char) : Bool
```

**def IndexString**[\[source\]](#)

*A string of indices to be associated with a tensor. For example,  $u_{\theta} u_{26\theta} u_3$ .*

```
def IndexString : Type
```

**def toCharList**[\[source\]](#)

*The character list associated with a index string.*

```
def toCharList (s : IndexString X) : List Char
```

**def charListToOptionIndex**[\[source\]](#)

*Takes a list of characters to the corresponding index if it exists else to none.*

```
def charListToOptionIndex (l : List Char) : Option (Index X)
```

**def toIndexList**[\[source\]](#)

*The indices associated to an index string.*

```
def toIndexList (s : IndexString X) : IndexList X
```

**def toIndexList'**[\[source\]](#)

*The formation of an index list from a string s satisfying listCharIndexStringBool.*

```
def toIndexList' (s : String) (hs : listCharIsIndexString X s.toList = true) : IndexList X
```

**def fromIndexString**[\[source\]](#)

*The construction of a tensor index from a tensor and a string satisfying conditions which are easy to check automatically.*

```
noncomputable def fromIndexString (T :  $\mathcal{T}$ .Tensor cn) (s : String)
  (hs : listCharIsIndexString  $\mathcal{T}$ .toTensorColor.Color s.toList = true)
  (hn : n = (toIndexList' s hs).length)
  (hD : IndexList.OnlyUniqueDuals (toIndexList' s hs))
  (hC : IndexList.ColorCond (toIndexList' s hs))
  (hd : TensorColor.ColorMap.DualMap (toIndexList' s hs).colorMap
```

```
(cn ◦ Fin.cast hn.symm)) :  $\mathcal{T}$ .TensorIndex
```

## 2.20 HepLean.SpaceTime.LorentzTensor.IndexNotation.TensorIndex

[HepLean/SpaceTime/LorentzTensor/IndexNotation/TensorIndex.lean](#) on GitHub.

### structure TensorIndex

[\[source\]](#)

*The structure an tensor with a index specification e.g.  $u_1 u_2$ .*

```
structure TensorIndex extends ColorIndexList  $\mathcal{T}$ .toTensorColor where
  tensor :  $\mathcal{T}$ .Tensor toColorIndexList.colorMap'
```

### instance Coe $\mathcal{T}$ .TensorIndex (ColorIndexList $\mathcal{T}$ .toTensorColor)

[\[source\]](#)

```
instance : Coe  $\mathcal{T}$ .TensorIndex (ColorIndexList  $\mathcal{T}$ .toTensorColor) where
```

### lemma colormap\_mapIso

[\[source\]](#)

```
lemma colormap_mapIso {T1 T2 :  $\mathcal{T}$ .TensorIndex} (hi : T1.toColorIndexList = T2.toColorIndexList)
  :
  ColorMap.MapIso (Fin.castOrderIso (congrArg IndexList.length (congrArg toIndexList hi))).
  toEquiv
  T1.colorMap' T2.colorMap'
```

### lemma ext

[\[source\]](#)

```
lemma ext {T1 T2 :  $\mathcal{T}$ .TensorIndex} (hi : T1.toColorIndexList = T2.toColorIndexList)
  (h : T1.tensor =  $\mathcal{T}$ .mapIso (Fin.castOrderIso (by simp [IndexList.length, hi])).toEquiv
  (colormap_mapIso hi.symm) T2.tensor) : T1 = T2
```

### lemma index\_eq\_of\_eq

[\[source\]](#)

```
lemma index_eq_of_eq {T1 T2 :  $\mathcal{T}$ .TensorIndex} (h : T1 = T2) :
  T1.toColorIndexList = T2.toColorIndexList
```

### def tensorIso

[\[source\]](#)

*Given an equality of TensorIndex, the isomorphism taking one underlying tensor to the other.*

```
@[simp]
def tensorIso {T1 T2 :  $\mathcal{T}$ .TensorIndex} (h : T1 = T2) :
   $\mathcal{T}$ .Tensor T2.colorMap'  $\approx_l[R]$   $\mathcal{T}$ .Tensor T1.colorMap'
```

### lemma tensor\_eq\_of\_eq

[\[source\]](#)

```
@[simp]
lemma tensor_eq_of_eq {T1 T2 :  $\mathcal{T}$ .TensorIndex} (h : T1 = T2) :
  T1.tensor = tensorIso h T2.tensor
```

**def mkDualMap**[\[source\]](#)

*The construction of a `TensorIndex` from a tensor, a `IndexListColor`, and a condition on the dual maps.*

```
def mkDualMap (T :  $\mathcal{T}$ .Tensor cn) (l : ColorIndexList  $\mathcal{T}$ .toTensorColor) (hn : n = l.1.length)
  (hd : ColorMap.DualMap l.colorMap' (cn  $\circ$  Fin.cast hn.symm)) :
   $\mathcal{T}$ .TensorIndex where
```

**def contr**[\[source\]](#)

*The contraction of indices in a `TensorIndex`.*

```
def contr (T :  $\mathcal{T}$ .TensorIndex) :  $\mathcal{T}$ .TensorIndex where
```

**lemma contr\_tensor**[\[source\]](#)

```
@[simp]
lemma contr_tensor (T :  $\mathcal{T}$ .TensorIndex) :
  T.contr.tensor = (( $\mathcal{T}$ .mapIso (Equiv.refl _) T.contrEquiv_colorMapIso <|
     $\mathcal{T}$ .contr T.toColorIndexList.contrEquiv T.contrEquiv_contrCond T.tensor))
```

**lemma contr\_of\_withDual\_empty**[\[source\]](#)

*Applying `contr` to a tensor whose indices has no contracts does not do anything.*

```
@[simp]
lemma contr_of_withDual_empty (T :  $\mathcal{T}$ .TensorIndex) (h : T.withDual =  $\emptyset$ ) :
  T.contr = T
```

**lemma contr\_tensor\_of\_withDual\_empty**[\[source\]](#)

```
lemma contr_tensor_of_withDual_empty (T :  $\mathcal{T}$ .TensorIndex) (h : T.withDual =  $\emptyset$ ) :
  T.contr.tensor = tensorIso (contr_of_withDual_empty T h) T.tensor
```

**lemma contr\_contr**[\[source\]](#)

```
@[simp]
lemma contr_contr (T :  $\mathcal{T}$ .TensorIndex) : T.contr.contr = T.contr
```

**lemma contr\_toColorIndexList**[\[source\]](#)

```
@[simp]
lemma contr_toColorIndexList (T :  $\mathcal{T}$ .TensorIndex) :
  T.contr.toColorIndexList = T.toColorIndexList.contr
```

**lemma contr\_toIndexList**[\[source\]](#)

```
lemma contr_toIndexList (T :  $\mathcal{T}$ .TensorIndex) :
  T.contr.toIndexList = T.toIndexList.contrIndexList
```

**def Rel**[\[source\]](#)

*An (equivalence) relation on two  $\mathcal{T}$ .TensorIndex. The point in this equivalence relation is that certain things (like the permutation of indices, the contraction of indices, or rising or lowering indices) can be placed in the indices or moved to the tensor itself. These two descriptions are equivalent.*

```
def Rel (T1 T2 :  $\mathcal{T}$ .TensorIndex) : Prop
```

**lemma Rel.refl**[\[source\]](#)

*Rel is reflexive.*

```
lemma refl (T :  $\mathcal{T}$ .TensorIndex) : Rel T T
```

**lemma Rel.symm**[\[source\]](#)

*Rel is symmetric.*

```
lemma symm {T1 T2 :  $\mathcal{T}$ .TensorIndex} (h : Rel T1 T2) : Rel T2 T1
```

**lemma Rel.trans**[\[source\]](#)

*Rel is transitive.*

```
lemma trans {T1 T2 T3 :  $\mathcal{T}$ .TensorIndex} (h1 : Rel T1 T2) (h2 : Rel T2 T3) : Rel T1 T3
```

**lemma isEquivalence**[\[source\]](#)

*Rel forms an equivalence relation.*

```
lemma isEquivalence : Equivalence (@Rel _ _  $\mathcal{T}$  _ _) where
```

**lemma to\_eq**[\[source\]](#)

*The equality of tensors corresponding to related tensor indices.*

```
lemma to_eq {T1 T2 :  $\mathcal{T}$ .TensorIndex} (h : Rel T1 T2) :  
  T1.contr.tensor =  $\mathcal{T}$ .mapIso (contrPermEquiv h.1).symm  
  (contrPermEquiv_colorMap_iso h.1) T2.contr.tensor
```

**lemma of\_withDual\_empty**[\[source\]](#)

```
lemma of_withDual_empty {T1 T2 :  $\mathcal{T}$ .TensorIndex} (h : T1.ContrPerm T2)  
  (h1 : T1.withDual =  $\emptyset$ ) (h2 : T2.withDual =  $\emptyset$ )  
  (hT : T1.tensor =  
     $\mathcal{T}$ .mapIso (permEquiv h h1 h2).symm (permEquiv_colorMap_iso h h1 h2) T2.tensor) : Rel T1 T2
```

**instance asSetoid**[\[source\]](#)

*The structure of a Setoid on  $\mathcal{T}$ .TensorIndex induced by Rel.*

```
instance asSetoid : Setoid  $\mathcal{T}$ .TensorIndex
```

**lemma rel\_contr**[\[source\]](#)

*A tensor index is equivalent to its contraction.*

```
lemma rel_contr (T :  $\mathcal{T}$ .TensorIndex) : T  $\approx$  T.contr
```

**instance SMul R  $\mathcal{T}$ .TensorIndex**[\[source\]](#)

*The scalar multiplication of a TensorIndex by an element of R.*

```
instance : SMul R  $\mathcal{T}$ .TensorIndex where
```

**lemma smul\_index**[\[source\]](#)

```
@[simp]
```

```
lemma smul_index (r : R) (T :  $\mathcal{T}$ .TensorIndex) : (r • T).toColorIndexList = T.toColorIndexList
```

**lemma smul\_tensor**[\[source\]](#)

```
@[simp]
```

```
lemma smul_tensor (r : R) (T :  $\mathcal{T}$ .TensorIndex) : (r • T).tensor = r • T.tensor
```

**lemma smul\_contr**[\[source\]](#)

```
@[simp]
```

```
lemma smul_contr (r : R) (T :  $\mathcal{T}$ .TensorIndex) : (r • T).contr = r • T.contr
```

**lemma smul\_rel**[\[source\]](#)

```
lemma smul_rel {T1 T2 :  $\mathcal{T}$ .TensorIndex} (h : T1  $\approx$  T2) (r : R) : r • T1  $\approx$  r • T2
```

**def AddCond**[\[source\]](#)

*The condition on tensors with indices for their addition to exists. This condition states that the the indices of one tensor are exact permutations of indices of another after contraction. This includes the id of the index and the color.*

*This condition is general enough to allow addition of e.g.  $\psi_{u1u2} + \phi_{u2u1}$ , but will NOT allow e.g.  $\psi_{u1u2} + \phi^{u2}_{u1}$ .*

```
def AddCond (T1 T2 :  $\mathcal{T}$ .TensorIndex) : Prop
```

**lemma to\_PermContr**[\[source\]](#)

```
lemma to_PermContr {T1 T2 :  $\mathcal{T}$ .TensorIndex} (h : AddCond T1 T2) :
```

```
T1.toColorIndexList.ContrPerm T2.toColorIndexList
```

**lemma AddCond.symm**[\[source\]](#)

```
@[symm]
```

```
lemma symm {T1 T2 :  $\mathcal{T}$ .TensorIndex} (h : AddCond T1 T2) : AddCond T2 T1
```

**lemma AddCond.refl**[\[source\]](#)

```
lemma refl (T :  $\mathcal{T}$ .TensorIndex) : AddCond T T
```

**lemma AddCond.trans**[\[source\]](#)

```
lemma trans {T1 T2 T3 :  $\mathcal{T}$ .TensorIndex} (h1 : AddCond T1 T2) (h2 : AddCond T2 T3) :  
  AddCond T1 T3
```

**lemma rel\_left**[\[source\]](#)

```
lemma rel_left {T1 T1' T2 :  $\mathcal{T}$ .TensorIndex} (h : AddCond T1 T2) (h' : T1 ≈ T1') :  
  AddCond T1' T2
```

**lemma rel\_right**[\[source\]](#)

```
lemma rel_right {T1 T2 T2' :  $\mathcal{T}$ .TensorIndex} (h : AddCond T1 T2) (h' : T2 ≈ T2') :  
  AddCond T1 T2'
```

**def addCondEquiv**[\[source\]](#)

*The equivalence between indices after contraction given a AddCond.*

```
@[simp]
```

```
def addCondEquiv {T1 T2 :  $\mathcal{T}$ .TensorIndex} (h : AddCond T1 T2) :  
  Fin T1.contr.length ≈ Fin T2.contr.length
```

**lemma addCondEquiv\_colorMap**[\[source\]](#)

```
lemma addCondEquiv_colorMap {T1 T2 :  $\mathcal{T}$ .TensorIndex} (h : AddCond T1 T2) :  
  ColorMap.MapIso (addCondEquiv h) T1.contr.colorMap' T2.contr.colorMap'
```

**def add**[\[source\]](#)

*The addition of two TensorIndex given the condition that, after contraction, their index lists are the same.*

```
def add (T1 T2 :  $\mathcal{T}$ .TensorIndex) (h : AddCond T1 T2) :  
   $\mathcal{T}$ .TensorIndex where
```

**lemma add\_right**[\[source\]](#)

```
lemma add_right {T1 T2 T3 :  $\mathcal{T}$ .TensorIndex} (h : AddCond T1 T3) (h' : AddCond T2 T3) :  
  AddCond T1 (T2 +[h'] T3)
```

**lemma add\_left**[\[source\]](#)

```
lemma add_left {T1 T2 T3 :  $\mathcal{T}$ .TensorIndex} (h : AddCond T1 T2) (h' : AddCond T2 T3) :  
  AddCond (T1 +[h] T2) T3
```

**lemma of\_add\_right'**[\[source\]](#)

```
lemma of_add_right' {T1 T2 T3 :  $\mathcal{T}$ .TensorIndex} {h' : AddCond T2 T3} (h : AddCond T1 (T2 +[h'] T3)) :
  AddCond T1 T3
```

**lemma of\_add\_right**[\[source\]](#)

```
lemma of_add_right {T1 T2 T3 :  $\mathcal{T}$ .TensorIndex} {h' : AddCond T2 T3} (h : AddCond T1 (T2 +[h'] T3)) :
  AddCond T1 T2
```

**lemma of\_add\_left**[\[source\]](#)

```
lemma of_add_left {T1 T2 T3 :  $\mathcal{T}$ .TensorIndex} {h' : AddCond T1 T2}
  (h : AddCond (T1 +[h'] T2) T3) : AddCond T2 T3
```

**lemma of\_add\_left'**[\[source\]](#)

```
lemma of_add_left' {T1 T2 T3 :  $\mathcal{T}$ .TensorIndex} {h' : AddCond T1 T2}
  (h : AddCond (T1 +[h'] T2) T3) : AddCond T1 T3
```

**lemma add\_left\_of\_add\_right**[\[source\]](#)

```
lemma add_left_of_add_right {T1 T2 T3 :  $\mathcal{T}$ .TensorIndex} {h' : AddCond T2 T3}
  (h : AddCond T1 (T2 +[h'] T3)) : AddCond (T1 +[of_add_right h] T2) T3
```

**lemma add\_right\_of\_add\_left**[\[source\]](#)

```
lemma add_right_of_add_left {T1 T2 T3 :  $\mathcal{T}$ .TensorIndex} {h' : AddCond T1 T2}
  (h : AddCond (T1 +[h'] T2) T3) : AddCond T1 (T2 +[of_add_left h] T3)
```

**lemma AddCond.add\_comm**[\[source\]](#)

```
lemma add_comm {T1 T2 :  $\mathcal{T}$ .TensorIndex} (h : AddCond T1 T2) :
  AddCond (T1 +[h] T2) (T2 +[h.symm] T1)
```

**lemma add\_toColorIndexList**[\[source\]](#)

```
@[simp]
lemma add_toColorIndexList (T1 T2 :  $\mathcal{T}$ .TensorIndex) (h : AddCond T1 T2) :
  (add T1 T2 h).toColorIndexList = T2.toColorIndexList.contr
```

**lemma add\_tensor**[\[source\]](#)

```
@[simp]
lemma add_tensor (T1 T2 :  $\mathcal{T}$ .TensorIndex) (h : AddCond T1 T2) :
  (add T1 T2 h).tensor =
  ( $\mathcal{T}$ .mapIso (addCondEquiv h) (addCondEquiv_colorMap h) T1.contr.tensor) + T2.contr.tensor
```

**lemma smul\_add**[\[source\]](#)*Scalar multiplication commutes with addition.*

```
lemma smul_add (r : R) (T1 T2 :  $\mathcal{T}$ .TensorIndex) (h : AddCond T1 T2) :
  r • (T1 +[h] T2) = r • T1 +[h] r • T2
```

**lemma add\_withDual\_empty**[\[source\]](#)

```
lemma add_withDual_empty (T1 T2 :  $\mathcal{T}$ .TensorIndex) (h : AddCond T1 T2) :
  (T1 +[h] T2).withDual = ∅
```

**lemma contr\_add**[\[source\]](#)

```
@[simp]
lemma contr_add (T1 T2 :  $\mathcal{T}$ .TensorIndex) (h : AddCond T1 T2) :
  (T1 +[h] T2).contr = T1 +[h] T2
```

**lemma contr\_add\_tensor**[\[source\]](#)

```
lemma contr_add_tensor (T1 T2 :  $\mathcal{T}$ .TensorIndex) (h : AddCond T1 T2) :
  (T1 +[h] T2).contr.tensor =
   $\mathcal{T}$ .mapIso (Fin.castOrderIso (by rw [index_eq_of_eq (contr_add T1 T2 h)])) .toEquiv
  (colormap_mapIso (by simp)) (T1 +[h] T2).tensor
```

**lemma TensorIndex.add\_comm**[\[source\]](#)

```
lemma add_comm {T1 T2 :  $\mathcal{T}$ .TensorIndex} (h : AddCond T1 T2) : T1 +[h] T2 ≈ T2 +[h.symm] T1
```

**lemma add\_rel\_left**[\[source\]](#)

```
lemma add_rel_left {T1 T1' T2 :  $\mathcal{T}$ .TensorIndex} (h : AddCond T1 T2) (h' : T1 ≈ T1') :
  T1 +[h] T2 ≈ T1' +[h.rel_left h'] T2
```

**lemma add\_rel\_right**[\[source\]](#)

```
lemma add_rel_right {T1 T2 T2' :  $\mathcal{T}$ .TensorIndex} (h : AddCond T1 T2) (h' : T2 ≈ T2') :
  T1 +[h] T2 ≈ T1 +[h.rel_right h'] T2'
```

**lemma add\_assoc'**[\[source\]](#)

```
lemma add_assoc' {T1 T2 T3 :  $\mathcal{T}$ .TensorIndex} {h' : AddCond T2 T3} (h : AddCond T1 (T2 +[h'] T3)) :
  T1 +[h] (T2 +[h'] T3) = T1 +[h'.of_add_right h] T2 +[h'.add_left_of_add_right h] T3
```

**lemma add\_assoc**[\[source\]](#)

```
lemma add_assoc {T1 T2 T3 :  $\mathcal{T}$ .TensorIndex} {h' : AddCond T1 T2} (h : AddCond (T1 +[h'] T2) T3) :
  T1 +[h'] T2 +[h] T3 = T1 +[h'.add_right_of_add_left h] (T2 +[h'.of_add_left h] T3)
```

**def ProdCond**[\[source\]](#)

*The condition on two `TensorIndex` which is true if and only if their `ColorIndexLists` are related by the condition `AppendCond`. That is, they can be appended to form a `ColorIndexList`.*

```
def ProdCond (T1 T2 :  $\mathcal{T}$ .TensorIndex) : Prop
```

**lemma to\_AppendCond**[\[source\]](#)

```
lemma to_AppendCond (h : ProdCond T1 T2) :  
  T1.AppendCond T2
```

**lemma ProdCond.symm**[\[source\]](#)

```
@[symm]  
lemma symm (h : ProdCond T1 T2) : ProdCond T2 T1
```

**def prod**[\[source\]](#)

*The tensor product of two `TensorIndex`.*

*Note: By default contraction is NOT done before taking the products.*

```
def prod (T1 T2 :  $\mathcal{T}$ .TensorIndex) (h : ProdCond T1 T2) :  $\mathcal{T}$ .TensorIndex where
```

**lemma prod\_toColorIndexList**[\[source\]](#)

```
@[simp]  
lemma prod_toColorIndexList (T1 T2 :  $\mathcal{T}$ .TensorIndex) (h : ProdCond T1 T2) :  
  (prod T1 T2 h).toColorIndexList = T1.toColorIndexList ++ [h] T2.toColorIndexList
```

**lemma prod\_toIndexList**[\[source\]](#)

```
@[simp]  
lemma prod_toIndexList (T1 T2 :  $\mathcal{T}$ .TensorIndex) (h : ProdCond T1 T2) :  
  (prod T1 T2 h).toIndexList = T1.toIndexList ++ T2.toIndexList
```

**2.21 HepLean.SpaceTime.LorentzTensor.Real.Basic**

[HepLean/SpaceTime/LorentzTensor/Real/Basic.lean](#) on GitHub.

**inductive ColorType**[\[source\]](#)

*The type colors for real Lorentz tensors.*

```
inductive ColorType  
| up  
| down
```

**def colorTypeEquivFin1Fin1**[\[source\]](#)

*An equivalence between `ColorType`  $\approx$  `Fin 1`  $\otimes$  `Fin 1`.*

```
def colorTypeEquivFin1Fin1 : ColorType ≈ Fin 1 ⊗ Fin 1 where
```

**instance DecidableEq ColorType**

[\[source\]](#)

```
instance : DecidableEq ColorType
```

**instance Fintype ColorType**

[\[source\]](#)

```
instance : Fintype ColorType where
```

**def realTensorColor**

[\[source\]](#)

*The color structure for real lorentz tensors.*

```
def realTensorColor : TensorColor where
```

**instance Fintype realTensorColor.Color**

[\[source\]](#)

```
instance : Fintype realTensorColor.Color
```

**instance DecidableEq realTensorColor.Color**

[\[source\]](#)

```
instance : DecidableEq realTensorColor.Color
```

## 2.22 HepLean.SpaceTime.LorentzTensor.Real.IndexNotation

[HepLean/SpaceTime/LorentzTensor/Real/IndexNotation.lean](#) on GitHub.

**instance IndexNotation realTensorColor.Color**

[\[source\]](#)

```
instance : IndexNotation realTensorColor.Color where
```

**instance IndexNotation (realLorentzTensor d).Color**

[\[source\]](#)

```
instance : IndexNotation (realLorentzTensor d).Color
```

**instance DecidableEq (realLorentzTensor d).Color**

[\[source\]](#)

```
instance : DecidableEq (realLorentzTensor d).Color
```

**lemma indexNotation\_eq\_color**

[\[source\]](#)

```
@[simp]
```

```
lemma indexNotation_eq_color : @realLorentzTensor.instIndexNotationColor d =  
  instIndexNotationColorRealTensorColor
```

**lemma decidableEq\_eq\_color**[\[source\]](#)

```
@[simp]
lemma decidableEq_eq_color : @realLorentzTensor.instDecidableEqColor d =
  instDecidableEqColorRealTensorColor
```

**lemma realLorentzTensor\_color**[\[source\]](#)

```
@[simp]
lemma realLorentzTensor_color : (realLorentzTensor d).Color = realTensorColor.Color
```

**lemma toTensorColor\_eq**[\[source\]](#)

```
@[simp]
lemma toTensorColor_eq : (realLorentzTensor d).toTensorColor = realTensorColor
```

**def fromIndexStringColor**[\[source\]](#)

*The construction of a tensor index from a tensor and a string satisfying conditions which can be automatically checked. This is a modified version of `TensorStructure.TensorIndex.mkDualMap` specific to real Lorentz tensors.*

```
noncomputable def fromIndexStringColor {cn : Fin n → realTensorColor.Color}
  (T : (realLorentzTensor d).Tensor cn) (s : String)
  (hs : listCharIsIndexString realTensorColor.Color s.toList = true)
  (hn : n = (toIndexList' s hs).length)
  (hD : (toIndexList' s hs).OnlyUniqueDuals)
  (hC : IndexList.ColorCond.bool (toIndexList' s hs))
  (hd : TensorColor.ColorMap.DualMap.boolFin
    (toIndexList' s hs).colorMap (cn ∘ Fin.cast hn.symm)) :
  (realLorentzTensor d).TensorIndex
```

**2.23 HepLean.SpaceTime.LorentzTensor.RisingLowering**

[HepLean/SpaceTime/LorentzTensor/RisingLowering.lean](#) on GitHub.

**def partDual**[\[source\]](#)

*Given an equivalence  $C \otimes P \simeq X$  the color map obtained by  $cX$  by dualising all indices in  $C$ .*

```
def partDual (e : C ⊗ P ≃ X) : C.ColorMap X
```

**def DualMap**[\[source\]](#)

*Two color maps are said to be dual if their quotients are dual.*

```
def DualMap (c1 c2 : C.ColorMap X) : Prop
```

**def boolFin**[\[source\]](#)

*The bool which if  $C.\text{colorQuot } (c_1 \ i) = C.\text{colorQuot } (c_2 \ i)$  is true for all  $i$ .*

```
def boolFin (c1 c2 : C.ColorMap (Fin n)) : Bool
```

**lemma boolFin\_DualMap**[\[source\]](#)

```
lemma boolFin_DualMap {c1 c2 : C.ColorMap (Fin n)} (h : boolFin c1 c2 = true) :
  DualMap c1 c2
```

**def boolFin'**[\[source\]](#)

*The bool which is true if  $C.colorQuot (c_1 i) = C.colorQuot (c_2 i)$  is true for all  $i$ .*

```
def boolFin' (c1 c2 : C.ColorMap (Fin n)) : Bool
```

**lemma boolFin'\_DualMap**[\[source\]](#)

```
lemma boolFin'_DualMap {c1 c2 : C.ColorMap (Fin n)} (h : boolFin' c1 c2 = true) :
  DualMap c1 c2
```

**lemma refl**[\[source\]](#)

```
lemma refl : DualMap c1 c1
```

**lemma symm**[\[source\]](#)

```
lemma symm (h : DualMap c1 c2) : DualMap c2 c1
```

**lemma trans**[\[source\]](#)

```
lemma trans (h : DualMap c1 c2) (h' : DualMap c2 c3) : DualMap c1 c3
```

**def split**[\[source\]](#)

*The splitting of  $X$  given two color maps based on the equality of the color.*

```
def split (c1 c2 : C.ColorMap X) : { x // c1 x ≠ c2 x } ⊕ { x // c1 x = c2 x } ≈ X
```

**lemma dual\_eq\_of\_neq**[\[source\]](#)

```
lemma dual_eq_of_neq (h : DualMap c1 c2) {x : X} (h' : c1 x ≠ c2 x) :
  C.τ (c1 x) = c2 x
```

**lemma split\_dual**[\[source\]](#)

```
@[simp]
```

```
lemma split_dual (h : DualMap c1 c2) : c1.partDual (split c1 c2) = c2
```

**lemma split\_dual'**[\[source\]](#)

```
@[simp]
```

```
lemma split_dual' (h : DualMap c1 c2) : c2.partDual (split c1 c2) = c1
```